

TOURIST DEMAND FORECASTING (SINGAPORE)

Project Title: Tourist Demand Forecasting

Case Study: 117 : A tourism organization wants to estimate demand for a destination or service over time.

Student Details:

- **Student Name:** Rayan Rawat
- **Cohort:** Mark Zuckerberg
- **Roll No:** 150096724141
- **Subject:** Machine Learning

Git-Hub Link:-

<https://github.com/Rayan-141/Tourist-Demand-Forecasting-Machine-Learning-Based-Monthly-Tourism-Demand-Forecasting->

TOURIST DEMAND FORECASTING FOR SINGAPORE

Using Machine Learning, Regression and Time-Series Forecasting

Document Control & Source Fidelity

Item	Project value
Project	Tourist Demand Forecasting for Singapore Using Machine Learning
Case Study	117
Dataset	International Visitor Arrivals By Place Of Residence, Monthly
Local dataset snapshot	January 2008 – January 2026
Raw shape	62 rows × 218 columns
Prepared target series	217 monthly observations
Modeling rows after feature creation	205
Chronological split	164 training / 41 test
Application	Streamlit
Core Python modules	data_utils.py, models.py, analysis.py, app.py
Evaluation	MAE, MSE, RMSE, R ² , TimeSeriesSplit

Official Dataset Source

Singapore Department of Statistics (SINGSTAT), original source: Singapore Tourism Board. Official data.gov.sg dataset page:

https://data.gov.sg/datasets/d_7e7b2ee60c6ffc962f80fef129cf306e/view

The official page identifies the dataset as monthly, reports the Jan 2008–Jan 2026 snapshot used here, and shows 62 rows and 218 columns.

Table of Contents

1. Executive Summary
 2. Problem Definition and Real-World Justification
 3. Project Objectives and Required Outcomes
 4. Machine-Learning Problem Formulation
 5. Dataset: Structure, Contents and Transformation
 6. Data Cleaning and Preprocessing
 7. Exploratory Data Analysis (EDA)
 8. Feature Engineering
 9. Core Machine-Learning Concepts Used
 10. Model Development
 11. Training, Test Split and Time-Series Cross-Validation
 12. Evaluation Metrics and Formulas
 13. Model Comparison and Final Interpretation
 14. Forecasting Logic and Recursive Prediction
 15. Streamlit Application and User Flow
 16. Project Files and Code Explanation
 17. Limitations, Risks and Future Improvements
 18. End-to-End Presentation/Demo Script
 19. Viva Question Bank with Answers
- Appendix A. Formula Sheet
- Appendix B. Source-Code Appendix
- Appendix C. Project Screenshots

1. Executive Summary

The project solves Case Study 117 by estimating future tourism demand for Singapore. The target is monthly international visitor arrivals. The project treats historical arrivals as a labelled numeric target, making the task supervised learning and regression; because observations are ordered in time and future values depend on earlier values, the same problem is also treated as time-series forecasting.

The supplied official-style dataset is wide: the DataSeries column contains 62 series and each month is a separate column. The project selects the row named “Total International Visitor Arrivals By Place Of Residence”, converts the monthly columns into a Date/Target time series, validates the data, performs EDA, creates lag/rolling/calendar features, and trains several regression approaches.

The implemented comparison contains Linear Regression, Polynomial Regression degree 2, Polynomial Regression degree 4, and Random Forest Regression. The current generated results show Linear Regression has the lowest test MAE and RMSE and the highest R^2 among the four experiments. TimeSeriesSplit is then used to check whether performance is stable across several historical validation windows.

The final Streamlit application exposes Home, EDA, Model Comparison and Forecast functionality. Forecasting is recursive: the first future prediction is fed back into the history so later future months can be predicted without knowing their actual arrivals in advance.

2. Problem Definition and Real-World Justification

2.1 Original problem statement

“A tourism organization wants to estimate demand for a destination or service over time. (With Proper Justification)”

2.2 Project-specific problem

Predict the number of international visitors arriving in Singapore for future months using historical monthly international visitor-arrival observations and engineered time-series features.

2.3 Why this is useful

- Tourism organizations can estimate expected demand levels for planning.
- Airports, transport operators, attractions and hospitality providers can use demand estimates for capacity planning.
- Staffing and service planning can be aligned with expected visitor volume.
- Historical seasonality can be incorporated into planning rather than relying only on a single average.
- Scenario analysis can be performed by changing the forecast horizon and model.

2.4 What the system actually predicts

Element	Meaning
Input X	Calendar variables + historical demand-derived features
Target y	Tourist_Arrivals: monthly international visitor arrivals
Output	Predicted visitor count for a future month
Task type	Supervised regression + time-series forecasting
Interface	Streamlit application

3. Project Objectives and Required Outcomes

Define the assigned real-world ML problem and justify why forecasting is appropriate. Identify and document a real tourism dataset and its source. Clean, transform and validate the data.

Perform EDA to understand trend, seasonality, distribution, anomalies and lag structure.

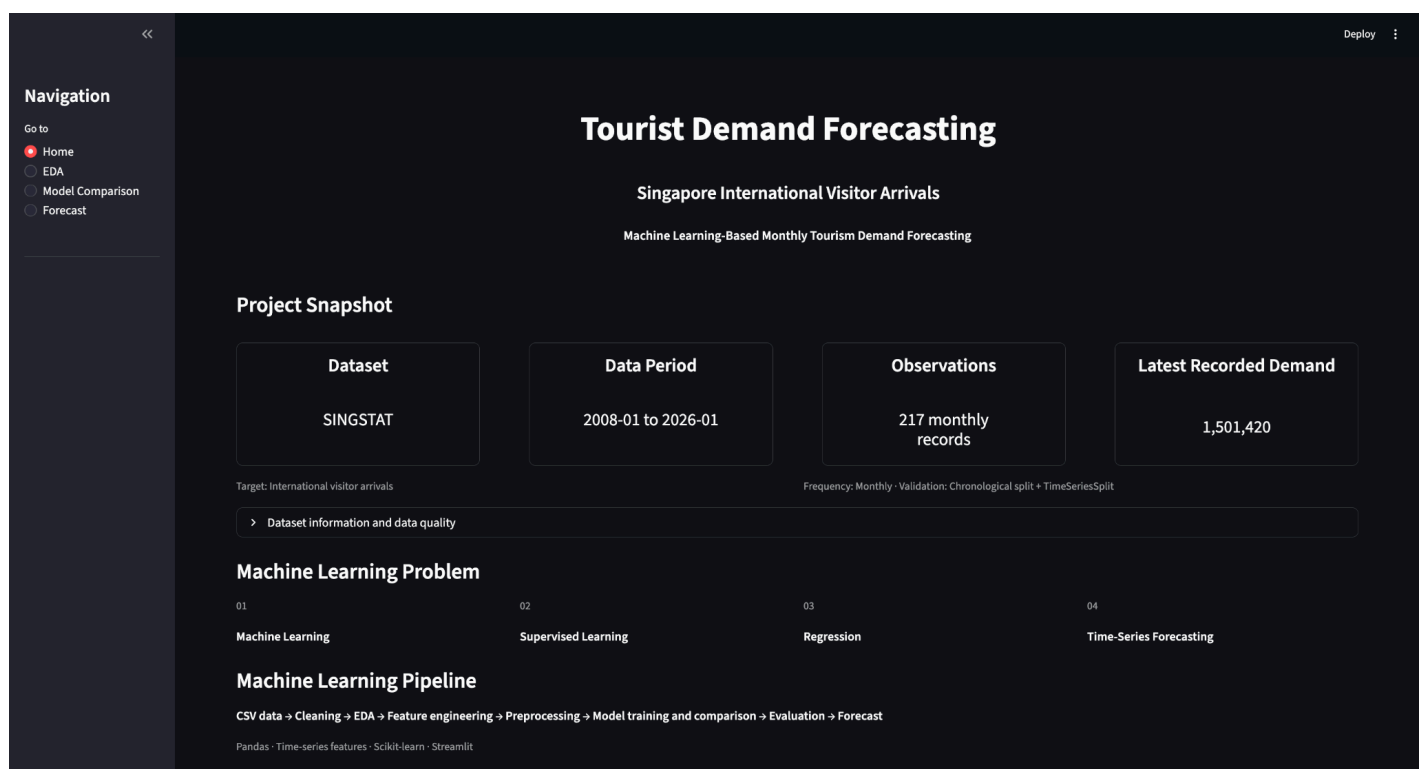
Create features that expose time and historical demand patterns to ML models , Train and compare meaningful regression models.

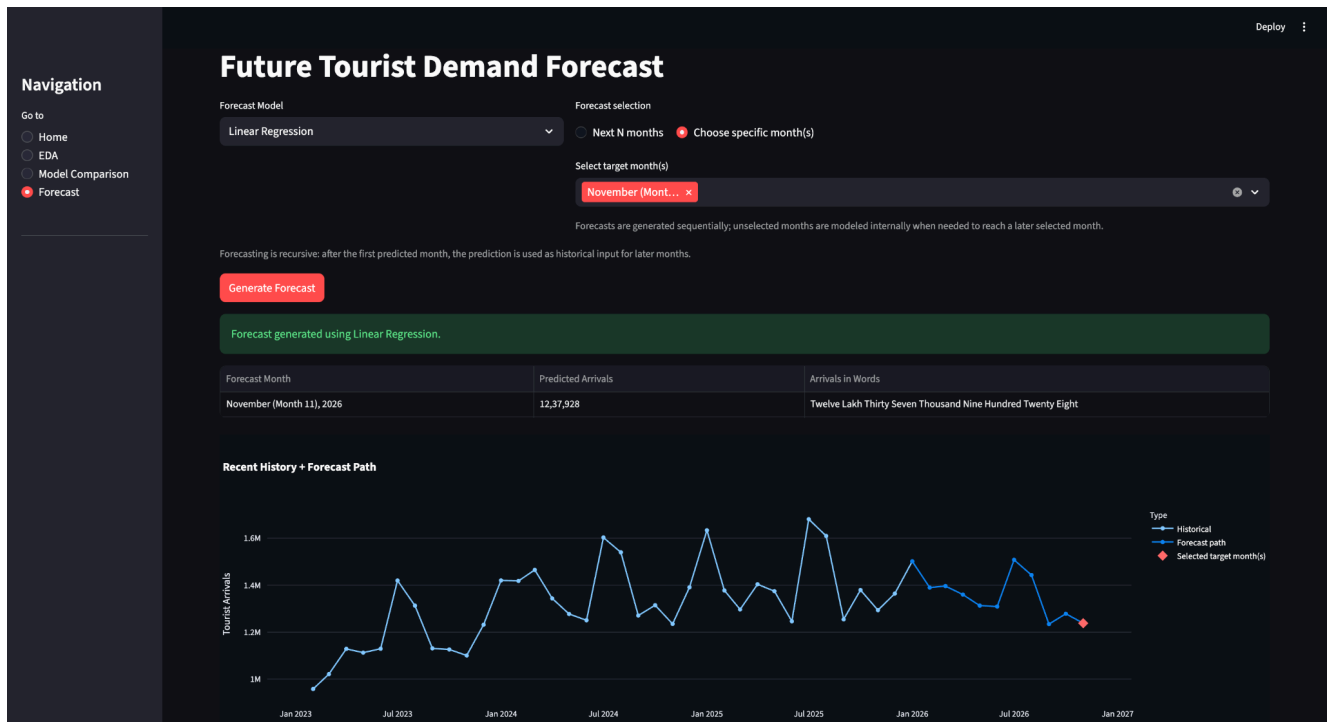
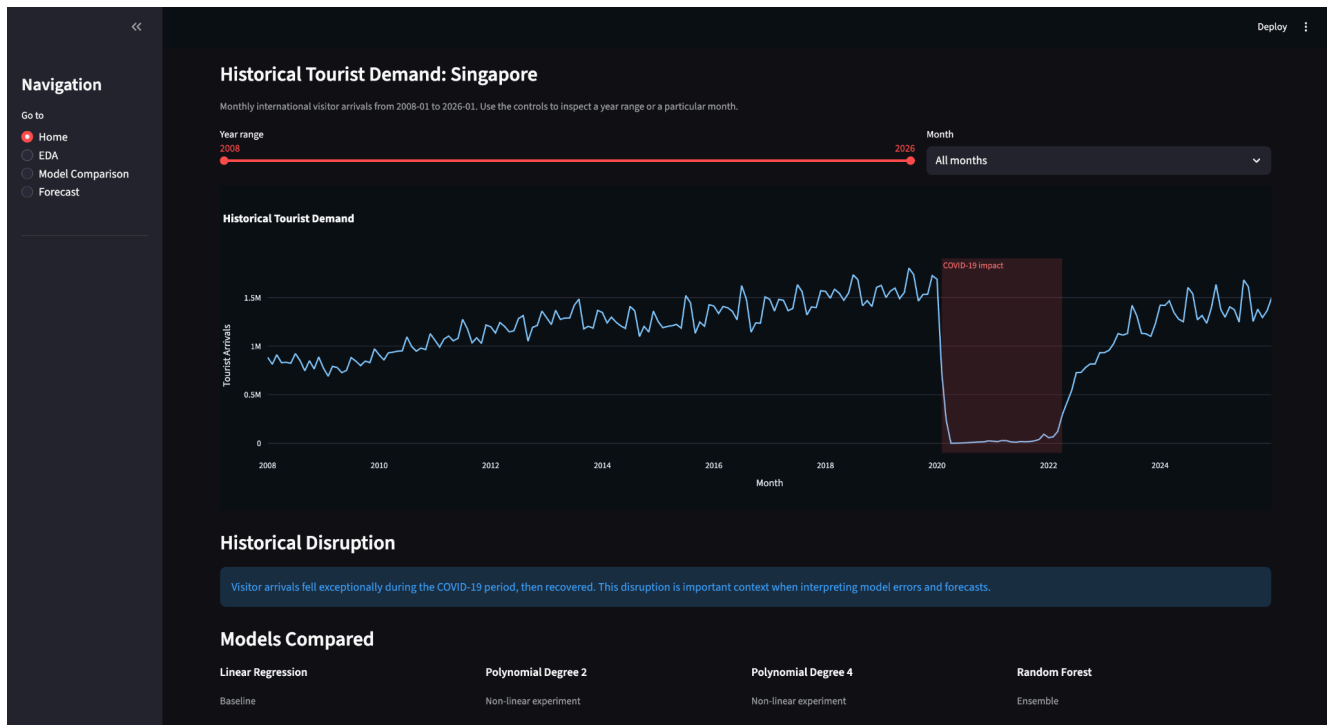
Evaluate predictions using MAE, MSE, RMSE and R².

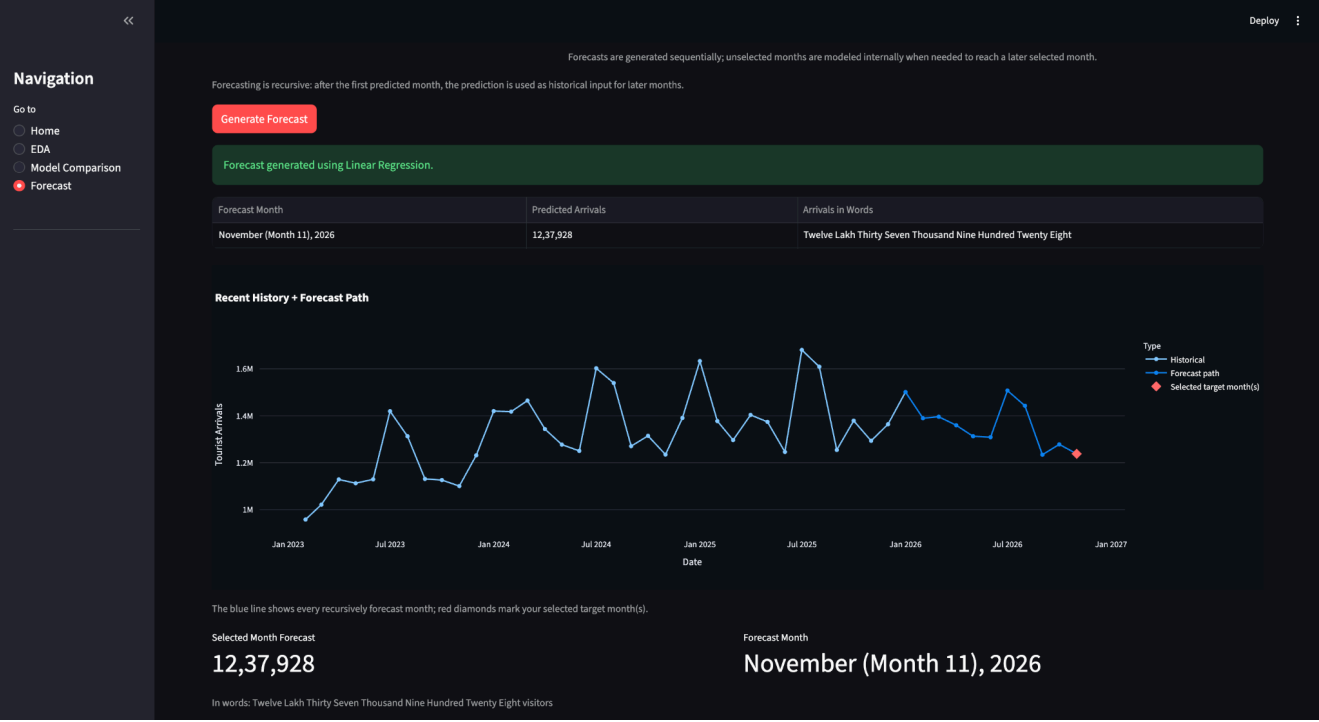
Use chronological validation rather than random shuffling.

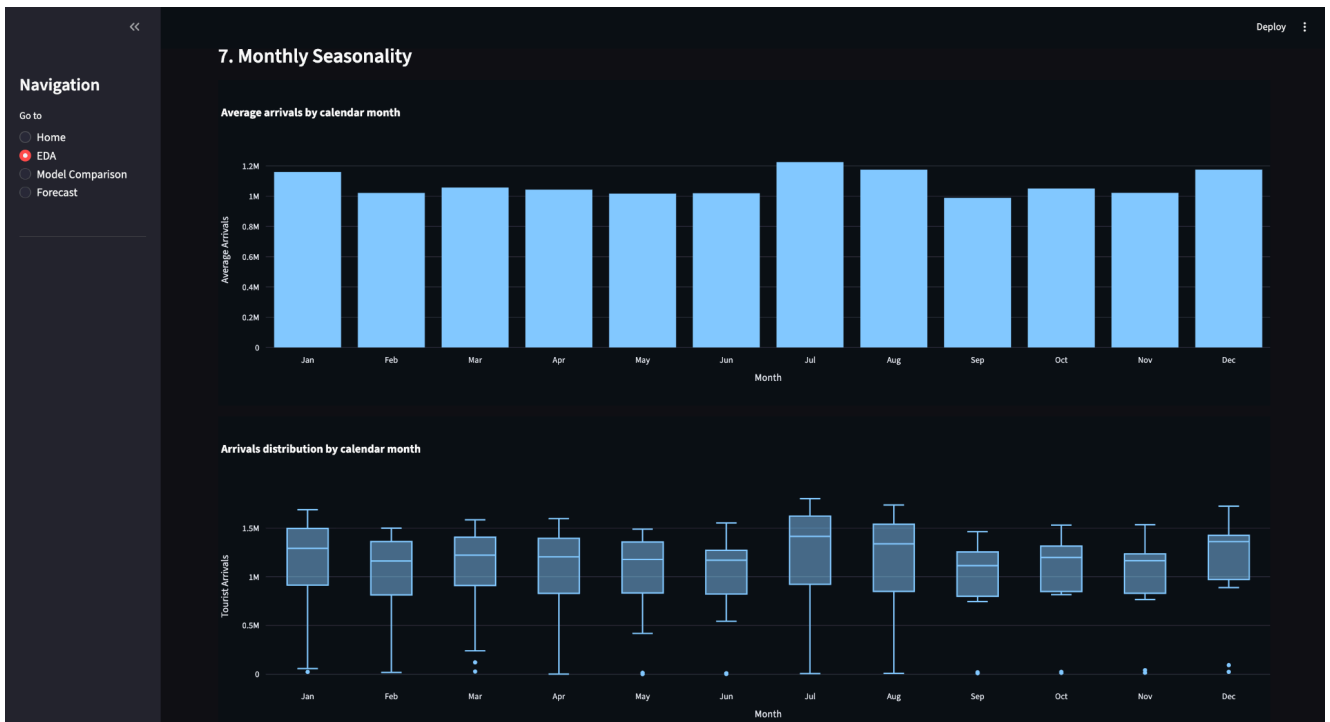
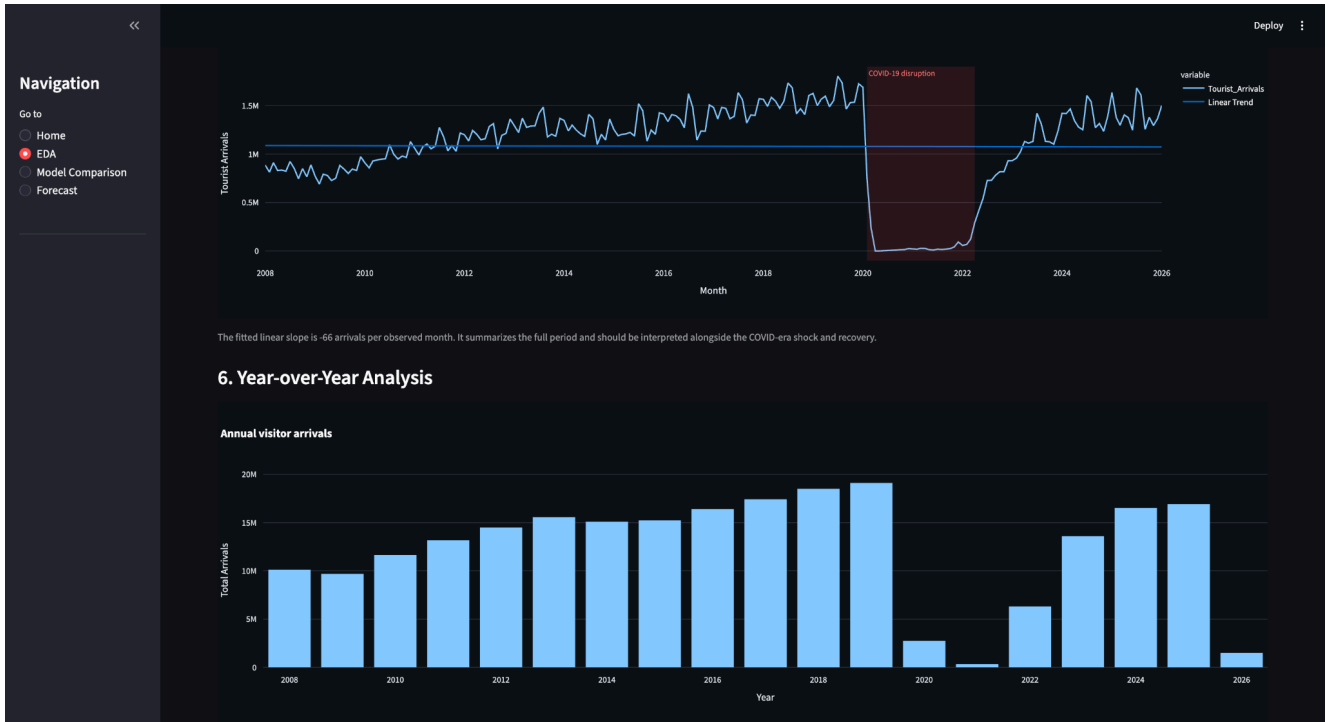
Perform error analysis through actual-vs-predicted views and cross-validation. Build a functional Streamlit prototype for interactive exploration and future forecasts.

ScreenShots:-



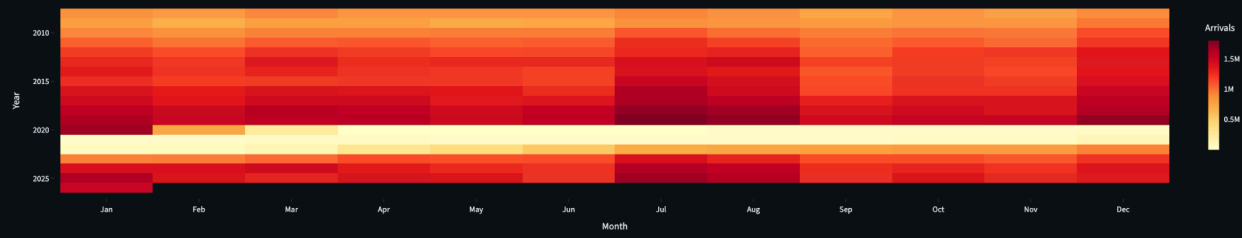






8. Year × Month Seasonality Heatmap

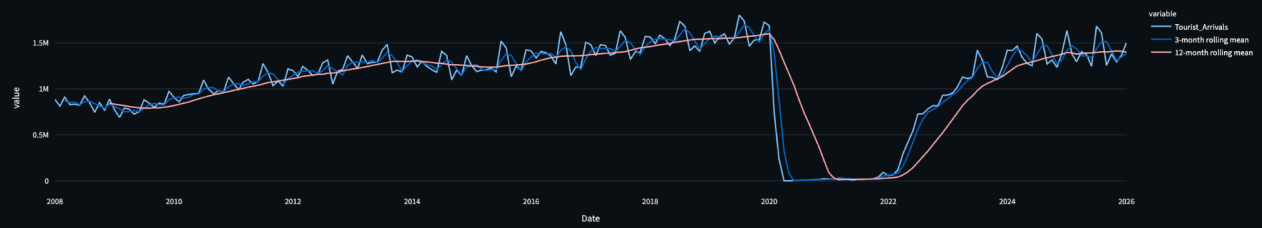
Monthly visitor arrivals by year



Blank cells indicate months not present in the source data; color encodes arrivals.

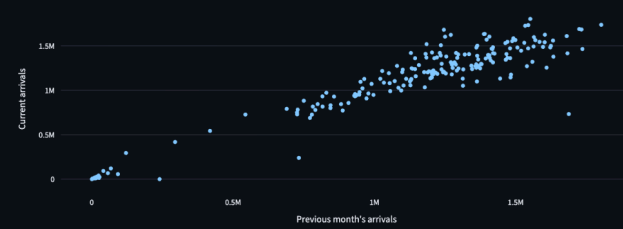
9. Rolling Averages

Observed arrivals with 3- and 12-month rolling means

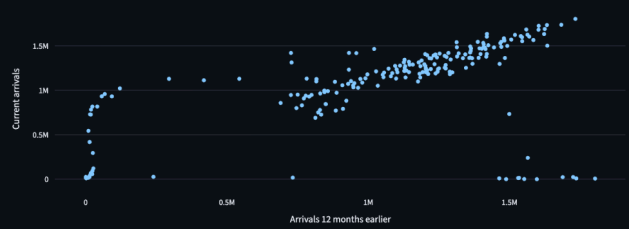


10. Lag Relationships and Autocorrelation

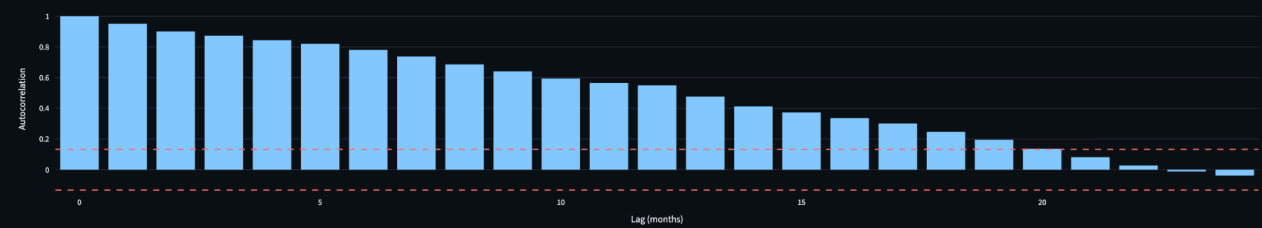
Current arrivals vs previous month



Current arrivals vs same month last year

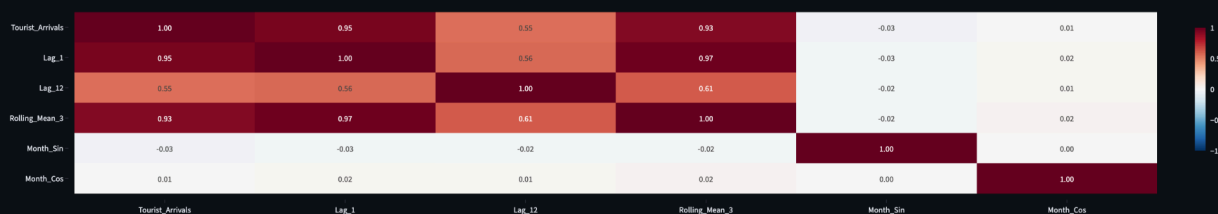


Autocorrelation by monthly lag



11. Feature Relationships

Target and selected feature correlations



Correlations describe pairwise linear association and do not establish causation.

12. EDA Findings and Feature-Engineering Rationale

- The mean of the latest 12 observations is +65.9% relative to the first 12, summarizing long-term change while averaging over individual months.
- The highest average calendar month is Jul (1,224,980); the lowest is Sep (988,557).
- Average arrivals during Feb 2020–Mar 2022 were 95.9% lower than the 2017–2019 average. This real-world disruption is retained, not treated as an automatic data error.
- Lag correlations are 0.951 at one month and 0.550 at 12 months; these help assess whether recent and annual history carry useful descriptive signal.
- The IQR rule flags 23 candidate months. Investigate context before deciding whether any value is erroneous; the EDA does not remove them.

EDA evidence	Related model features
Month averages range from 988,557 to 1,224,980 arrivals.	Month_Sin, Month_Cos, Month, Quarter
One-month autocorrelation is 0.951.	Lag_1, Lag_2, Lag_3
Twelve-month autocorrelation is 0.550.	Lag_12
Rolling means provide a smoothed summary of recent levels.	Rolling_Mean_3, Rolling_Mean_6, Rolling_Mean_12

The baseline, polynomial trend experiments, and Random Forest are compared because the series contains changing trend, monthly patterns, lag structure, and a major external shock. Chronological test results and TimeSeriesSplit—not EDA alone—determine comparative model performance.

3.1 Deliverable mapping

Requirement	Implemented component
Problem definition	Singapore monthly tourist-demand forecasting
Dataset	SINGSTAT / Singapore Tourism Board dataset
EDA	Trend, distribution, outliers, seasonality, heatmap, rolling means, autocorrelation, correlations
Preprocessing	Wide-to-long, dates, numeric conversion, duplicate/missing checks
Feature engineering	Calendar, cyclical month, lags, rolling means
Models	Linear, Polynomial 2/4, Random Forest
Evaluation	MAE, MSE, RMSE, R^2 , TimeSeriesSplit
Application	Streamlit Home / EDA / Model Comparison / Forecast

4. Machine-Learning Problem Formulation

4.1 Machine Learning

Machine learning means learning a useful relationship from data instead of manually writing every prediction rule. In this project, the algorithm learns how historical demand-related features relate to the known monthly arrival target.

4.2 Supervised Learning

Supervised learning uses examples where the target is already known during training. Here, each historical month has a known Tourist_Arrivals value. The model learns from X features and y target values.

Example: if January 2025 has known arrivals, its previous-month, previous-year, rolling and calendar features can be used to learn the mapping to the January 2025 target.

4.3 Regression

Regression predicts a continuous numeric quantity. Visitor arrivals are counts but are treated here as a continuous numeric target for regression. The output can be 1,389,780 visitors, 1,396,011 visitors, etc., rather than a class label.

4.4 Time-series forecasting

A time series is a sequence of observations indexed by time. Forecasting means estimating future observations while respecting temporal order. This project is therefore both a supervised regression problem and a time-series forecasting problem.

4.5 X and y

Term	Project meaning
------	-----------------

X	Feature matrix. Each row represents one month after feature engineering.
y	Target vector: Tourist_Arrivals.
X_train / y_train	Earlier chronological observations used to learn model parameters.
X_test / y_test	Latest chronological observations held out for final test evaluation.
Forecast X_next	Feature row constructed for a future month using calendar values and recent history.

5. Dataset: Structure, Contents and Transformation

5.1 Dataset identity

Property	Value
Dataset name	International Visitor Arrivals By Place Of Residence, Monthly
Publisher	Singapore Department of Statistics (SINGSTAT)
Original source	Singapore Tourism Board
Local snapshot	Jan 2008 to Jan 2026
Raw rows	62
Raw columns	218
Monthly columns	217
Prepared observations	217
Target row	Total International Visitor Arrivals By Place Of Residence

5.2 What was actually inside the raw dataset?

The 62 DataSeries entries include the overall total, regional groups, countries/markets and an Others category. Examples include Southeast Asia, Greater China, North Asia, South Asia, West Asia, Americas, Europe, Oceania,

Africa, plus countries such as Indonesia, Malaysia, China, India, Japan, South Korea, USA, UK, Australia and others.

The important point is that these rows are different series, not 62 independent observations of the ML target. The project therefore does not sum all 62 rows. It selects the official total row directly.

5.3 Why select the total row?

Critical viva point

If the official total row already contains total international visitor arrivals, summing the country/region rows again can double-count totals or aggregate hierarchical categories. The project therefore uses the provided total series as the target.

5.4 Why convert wide data to long format?

The raw file stores time horizontally: each month is a separate column. Machine-learning and time-series operations are much easier when one row represents one time point. The transformed structure is Date | Tourist_Arrivals.

5.5 What if we did NOT convert it?

- Every month would remain a separate column, so lag operations would have to manually reference columns.
- Rolling windows would be awkward because the time axis is not the row index.
- Plotting a standard Date-versus-arrivals time series would require an extra reshape step anyway.
- TimeSeriesSplit expects ordered samples as rows.
- Feature engineering and future recursive forecasting become harder to write and maintain.
- One row per month makes the dataset compatible with pandas/scikit-learn workflows.

5.6 Meaning of 217 monthly observations

The period is January 2008 through January 2026 inclusive. That is 18 complete years ($18 \times 12 = 216$ months) plus January 2026 = 217 monthly observations.

After creating a 12-month lag and 12-month rolling feature, the first 12 rows do not have all required historical features. They are dropped, leaving 205 modeling rows.

Stage	Count / Period
Raw monthly observations	217
Rows lost to initial lag/rolling requirements	12
Modeling rows	205
80% chronological training rows	164
20% chronological test rows	41
Training period	2009-01 to 2022-08
Test period	2022-09 to 2026-01

6. Data Cleaning and Preprocessing

6.1 Quality checks actually implemented

Check	Result
Raw rows	62
Raw columns	218
Raw missing cells	0
Raw duplicate rows	0
Series rows	217
Series missing values	0
Duplicate dates	0
Start	2008-01
End	2026-01

6.2 Cleaning operations

Read the CSV with pandas.

Find the exact target DataSeries row.

Convert month column names such as 2026Jan to datetime.
Convert arrival values to numeric using `pd.to_numeric(..., errors='coerce')`.
Drop rows where Date or Tourist_Arrivals is missing.
Drop duplicate dates.
Sort chronologically and reset the index.
Create time and historical-demand features.
Drop rows with NaN features created by lag/rolling windows.

6.3 Missing value handling

The raw dataset has 0 missing cells according to the project's quality report, and the prepared total series has 0 missing target values. NaNs introduced by lag/rolling feature creation are not original data errors; they are expected because a historical window does not exist for the earliest months.

6.4 Why not replace missing values with zero?

A missing value means unavailable/undefined, whereas zero means an observed zero. Replacing an unavailable historical window with zero would create artificial information and distort the model.

6.5 Scaling / standardization

The Linear Regression pipeline uses `StandardScaler` on numeric features. Standardization transforms a numeric feature approximately as $z = (x - \text{mean}) / \text{standard deviation}$. It puts numeric features on a comparable scale and is particularly useful for polynomial features and models sensitive to feature magnitude.

Random Forest does not require standardization because tree splits are based on thresholds rather than distances or coefficient magnitudes. The current code trains Random Forest directly on the numeric/categorical feature values.

6.6 One-hot encoding

Month and Quarter are categorical calendar features for the Linear Regression pipeline. OneHotEncoder converts categories into binary indicator columns. This avoids forcing the model to interpret month 12 as simply 'twice' month 6.

7. Exploratory Data Analysis (EDA)

EDA is the systematic inspection of data before and during modeling. In this project the EDA is not decorative; it provides evidence for feature choices and helps identify structural changes that can affect model errors.

7.1 EDA checklist used by the application

Dataset overview

Missing values and duplicates

Descriptive statistics

Distribution and outlier analysis

Time-series trend and historical demand

Year-over-year analysis

Monthly seasonality

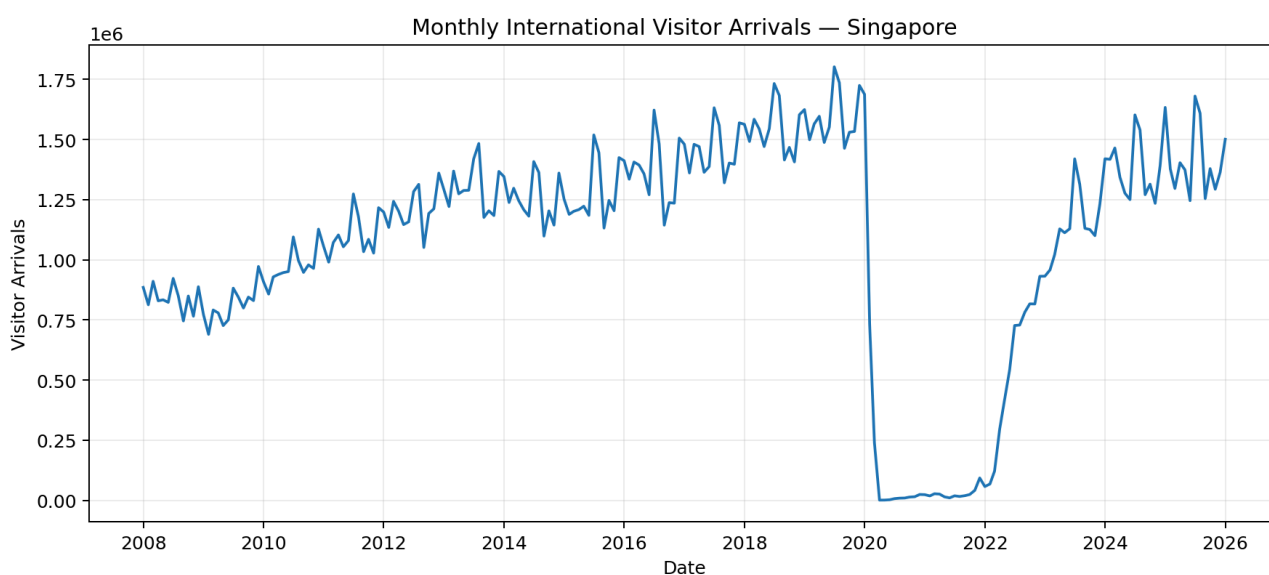
Year $\times$ Month seasonality heatmap

Rolling averages

Lag relationships and autocorrelation

Feature relationships

EDA findings and feature-engineering rationale

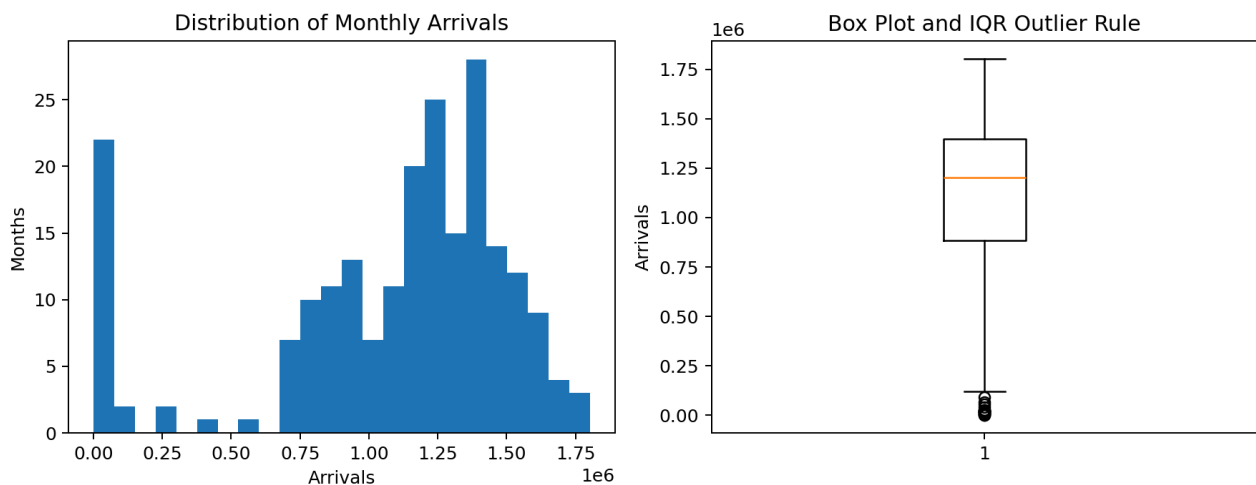


7.2 Distribution and outlier analysis

The overall mean is 1,080,070, median is 1,203,746, standard deviation is 460,026, minimum is 750, and maximum is 1,802,593 visitors.

The application uses the $1.5 \times \text{IQR}$ rule. $Q1 = 885,290$, $Q3 = 1,397,330$, $\text{IQR} = 512,040$; lower fence = 117,230; upper fence = 2,165,390. It flags 23 candidate outlier months.

These candidates are not automatically deleted. Most flagged months are in the 2020–2022 disruption period, so they represent a real structural change in the observed tourism series rather than a simple data-entry error.



7.3 What is an outlier?

An outlier is an observation unusually far from the typical distribution. In this project, an outlier is a candidate for investigation, not automatically a bad row. A pandemic-era collapse is economically meaningful even if statistically extreme.

7.4 Trend analysis

The long historical series shows a strong pre-2020 level, a sharp 2020–2021 collapse, and a substantial recovery from 2022 onward. The application highlights the 2020-02 to 2022-03 period as a disruption window.

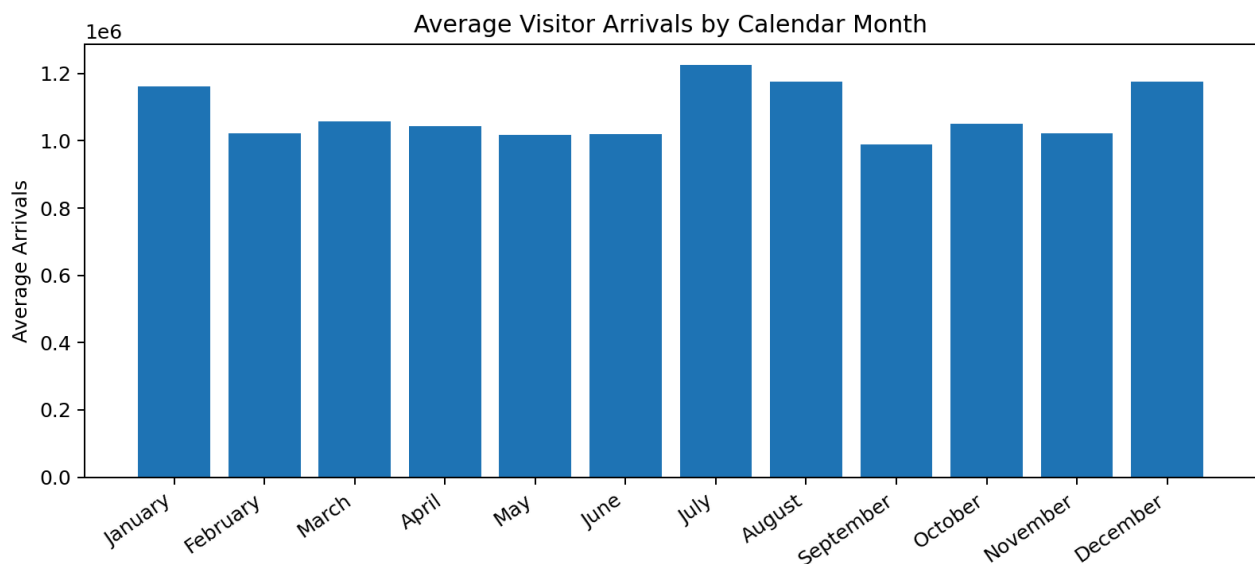
7.5 Year-over-year analysis

Year	Annual_Total	Monthly_Average	Months_Observed
2,019.00	19,116,016.00	1,593,001.33	12.0000

2,020.00	2,742,443.00	228,536.92	12.0000
2,021.00	330,059.00	27,504.92	12.0000
2,022.00	6,305,744.00	525,478.67	12.0000
2,023.00	13,602,215.00	1,133,517.92	12.0000
2,024.00	16,526,344.00	1,377,195.33	12.0000
2,025.00	16,912,288.00	1,409,357.33	12.0000
2,026.00	1,501,420.00	1,501,420.00	1.0000

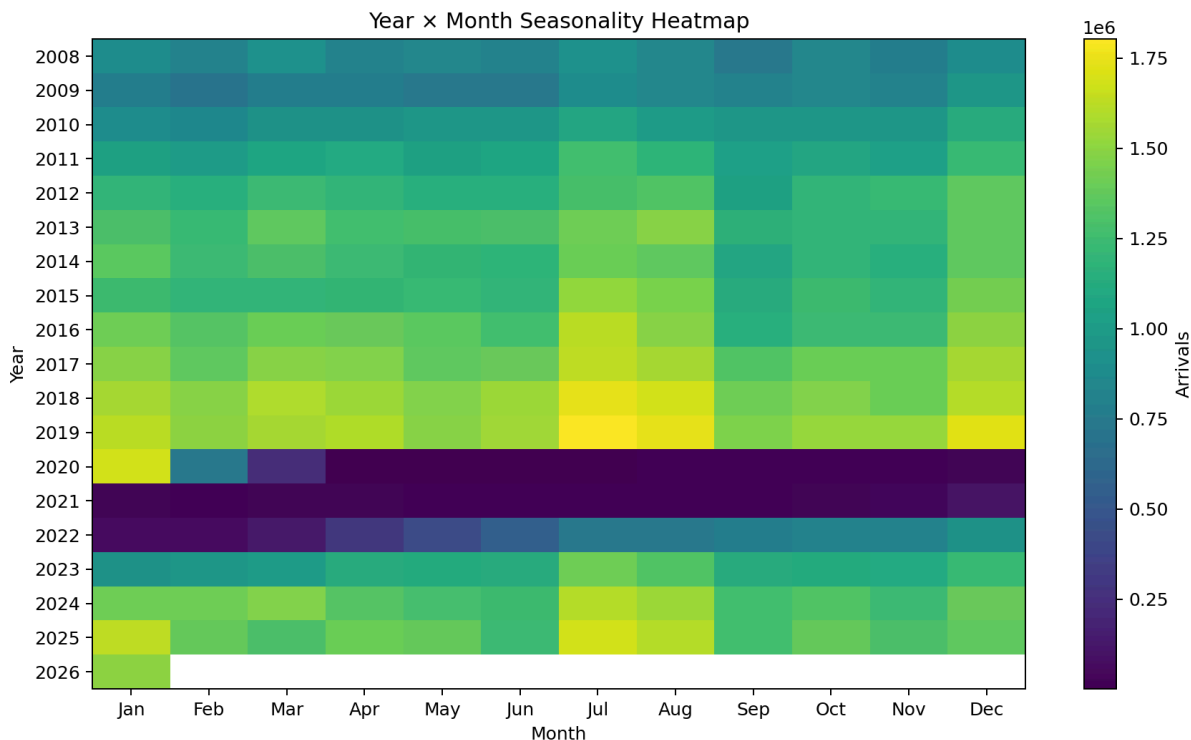
Important: 2026 currently contains only one observed month in the supplied snapshot, so its annual total is a partial-year total and must not be compared directly with complete years.

7.6 Monthly seasonality



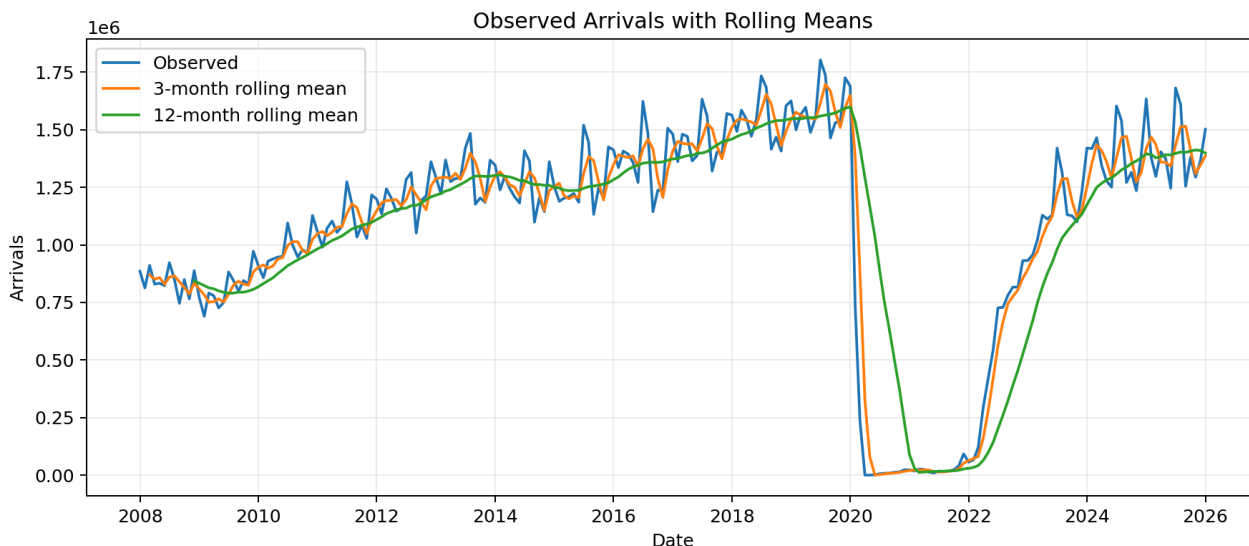
Monthly grouping helps answer: 'Do certain calendar months systematically have different demand levels?' The project computes mean, median, standard deviation and observation count for each month.

7.7 Year × Month seasonality heatmap



The heatmap makes two dimensions visible simultaneously: long-term changes by year and recurring calendar patterns by month.

7.8 Rolling averages



A rolling mean is a moving average. For a window k , $RM_k(t)$ = average of the previous k observations. The app's EDA uses rolling means directly for visualization; the model feature code uses a shifted rolling mean so the current target is excluded from its own feature.

7.9 Lag relationships

A lag is a previous value of the same time series. Lag 1 means the previous month; Lag 12 means the same calendar month one year earlier.

The project examines current arrivals against Lag 1 and Lag 12 because these reveal short-term dependence and annual seasonality.

7.10 Autocorrelation

Autocorrelation measures the correlation between a time series and a lagged version of itself. In the supplied data, lag-1 autocorrelation is 0.951 and lag-12 autocorrelation is 0.550. This supports the idea that recent months and year-ago months contain useful predictive information.

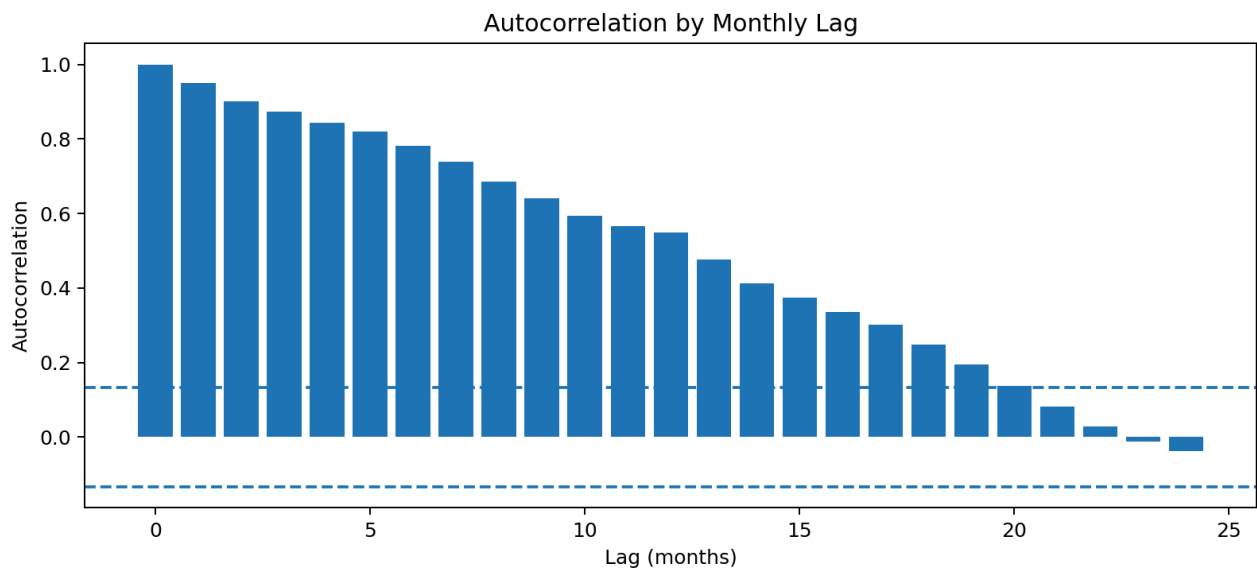
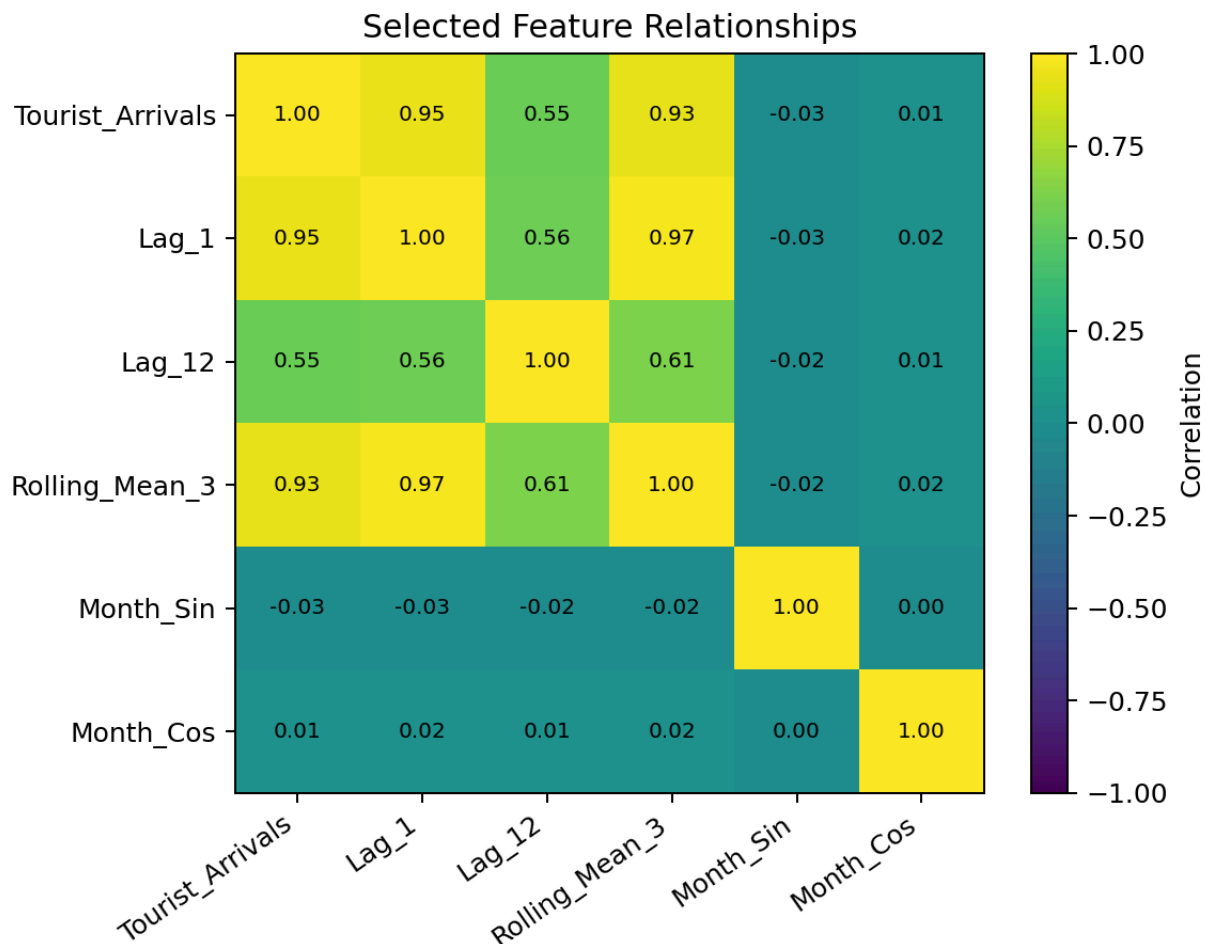


Figure 10. Autocorrelation by monthly lag.

7.11 Feature relationships



Correlation describes pairwise linear association; it does not prove causation. High correlation between Lag 1 and the target means recent demand tends to move with current demand, not that Lag 1 causes tourism.

7.12 Standard deviation, median and mode in EDA

Statistic	Formula / definition	Project use
Mean	$\Sigma x/n$	Arithmetic average; used in rolling means, summaries and CV means.
Median	Middle ordered value	Robust center when extreme values exist.
Mode	Most frequent value	Usually less informative for continuous monthly arrivals.
Standard deviation	$\sqrt{(\Sigma(x-\text{mean})^2)/(n-1))}$	Measures spread/variability.

Variance	Standard deviation ²	Squared spread; appears conceptually in error calculations.
IQR	Q3–Q1	Middle 50% spread; used for the 1.5×IQR outlier rule.

8. Feature Engineering

Feature engineering converts raw Date + Tourist_Arrivals into variables that the ML models can use to learn temporal patterns.

Feature	How calculated	Why used
Year	Calendar year	Long-term trend/regime changes
Month	1–12 calendar month	Monthly seasonality; one-hot encoded for Linear Regression
Quarter	1–4	Quarter-level seasonality; one-hot encoded for Linear Regression
Month_Sin	$\sin(2\pi \cdot \text{Month}/12)$	Cyclical representation of month
Month_Cos	$\cos(2\pi \cdot \text{Month}/12)$	Completes cyclical representation
Lag_1	$y(t-1)$	Immediate previous-month demand
Lag_2	$y(t-2)$	Short-term two-month dependency
Lag_3	$y(t-3)$	Short-term/quarterly recurrence
Lag_6	$y(t-6)$	Half-year recurrence
Lag_12	$y(t-12)$	Same month in previous year; annual seasonality
Rolling_Mean_3	mean of previous 3	Short-term smoothed level

Rolling_Mean_6	mean of previous 6	Medium-term smoothed level
Rolling_Mean_12	mean of previous 12	Annual baseline / smoothed level

8.1 Why shift before rolling?

The model feature code uses `Tourist_Arrivals.shift(1).rolling(window).mean()`. The shift is important: the rolling window must use only information available before the target month. If the current target were included, the feature would leak the answer into the input.

8.2 Cyclical encoding

January and December are adjacent in the calendar, but numeric month values 1 and 12 look far apart to a simple linear model. Sine/cosine encode the month around a circle. The current code uses both `Month_Sin` and `Month_Cos`.

8.3 Example

For a hypothetical March target, `Lag_1` is February's arrivals, `Lag_12` is March of the previous year, `Rolling_Mean_3` averages the previous three months, `Year` is the calendar year, and `Month_Sin/Month_Cos` represent March's location on the annual cycle.

9. Core Machine-Learning Concepts Used

Concept	Simple meaning
Feature	An input variable used by a model. Example: Lag_1.
Target	The value the model predicts. Here: Tourist_Arrivals.
Sample / observation	One modeling row representing one month.
Training	Learning model parameters from historical data.
Prediction	Applying the learned model to unseen feature values.
Generalization	How well the model works on unseen future-like data.
Overfitting	Model fits training data too closely and performs poorly on new data.
Underfitting	Model is too simple to capture useful structure.
Data leakage	Information from the future/target leaks into training features or preprocessing.
Hyperparameter	A model setting chosen before/during training, e.g. Random Forest <code>n_estimators=300</code> .
Pipeline	A sequence that keeps preprocessing and model steps together.
Ensemble	Combining multiple models/trees to produce a stronger prediction.
Cross-validation	Repeated train/validation experiments to assess stability.
Forecast horizon	How many future months are predicted.

9.1 What is a regression problem?

Regression maps features to a numeric value. In this project: $X \rightarrow$ predicted number of visitors. A classification model would instead output categories such as High/Low or Yes/No, which is not the requested target.

9.2 Why not classification?

There is no class label in the target. The target is an actual visitor count, and preserving the numeric magnitude is important.

9.3 Why not unsupervised learning?

Unsupervised learning does not require a known target. This project explicitly has historical known arrival values and wants to predict future arrival values, so supervised learning is appropriate.

10. Model Development

10.1 Linear Regression

Linear Regression models the target as a weighted combination of features: $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$. The coefficients are learned by minimizing the sum of squared residuals.

Why used here: it is a simple, interpretable baseline; it works naturally with engineered lag/calendar features; it is fast; and the current experiment shows strong test performance.

Important viva distinction: Linear Regression is not inherently a time-series model. It becomes a forecasting approach here because the input features contain time, lag and rolling information.

10.2 How Linear Regression calculates a prediction

For a future row, the pipeline preprocesses the numeric and categorical features, then the learned linear model calculates a weighted sum plus an intercept. Exact coefficients are generated from the fitted training data; they should not be invented in a viva.

10.3 Polynomial Regression

Polynomial Regression is implemented as PolynomialFeatures + StandardScaler + LinearRegression. The underlying regression is still linear in its learned coefficients, but the inputs are transformed into powers/interactions.

Degree	Concept	Interpretation
Degree 1	$\beta_0 + \beta_1x$	Straight line; effectively ordinary linear regression for one variable.
Degree 2	$\beta_0 + \beta_1x + \beta_2x^2$	Adds curvature.
Degree 3	Adds β_3x^3	More flexible; can model more complex curvature.
Degree 4	Adds β_4x^4	Even more flexible; can become unstable/overfit, especially for extrapolation.

10.4 Why degree 2 and degree 4?

The current code explicitly compares degrees [2, 4] as a simple syllabus-aligned time-trend curve-fitting experiment. Degree 1 is redundant with Linear Regression, so it is not included as a separate experiment. Degree 3 is also not included in the displayed experiment set; it could be tested, but its performance would need to be measured rather than assumed.

10.5 What happened to polynomial models?

Degree 2 and degree 4 extrapolate poorly on the held-out future period. Their R^2 values are strongly negative and their errors are millions of visitors. This is consistent with unstable high-order trend extrapolation on a series with a major structural disruption.

10.6 Random Forest Regression

Random Forest is an ensemble of decision trees. Each tree learns nonlinear threshold-based relationships; the forest averages the tree predictions for regression.

Why used: it provides a nonlinear tabular benchmark and can capture interactions between lags, calendar features and rolling statistics without requiring a single global linear equation.

Where used in this project: it is trained on the same 13 features as Linear Regression and is included in test-set comparison, TimeSeriesSplit, feature-importance analysis and the Forecast page.

Important limitation: tree ensembles do not naturally extrapolate smooth trends beyond the training range as a classical time-series trend model might. Recursive multi-step forecasting can also accumulate error.

10.7 Random Forest settings used

Parameter	Value	Purpose
n_estimators	300	Number of trees
max_depth	10	Maximum depth of each tree
min_samples_leaf	2	Minimum samples per leaf
random_state	42	Reproducibility
n_jobs	-1	Use available CPU workers

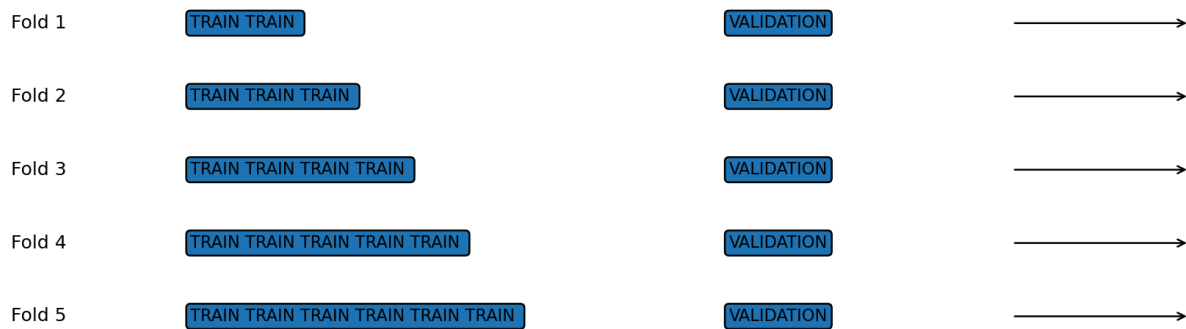
11. Training, Test Split and Time-Series Cross-Validation

11.1 Chronological train/test split

The exact project code uses test_size=0.20 and splits by row position, not randomly. Of 205 modeling rows, 164 are used for training and 41 for the final test. Training ends at 2022-08 and testing begins at 2022-09.

Why not random train_test_split? A random shuffle could place later months into training while earlier months are in testing. That can expose the model to future information and make the evaluation less realistic.

11.2 TimeSeriesSplit



Time moves left → right; validation is always later than training.

The project uses `TimeSeriesSplit(n_splits=5)` for Linear Regression and Random Forest. Each fold trains on earlier observations and validates on a later block. This is appropriate for ordered data because it respects the direction of time.

11.3 Cross-validation

Cross-validation repeats the training/validation process across multiple historical windows. The project records MAE, MSE, RMSE and R^2 for every fold, then summarizes mean and standard deviation for MAE, RMSE and R^2 .

11.4 Mean and standard deviation of CV

Mean tells the average performance across folds. Standard deviation tells how much the metric changes from fold to fold. A large standard deviation indicates performance is less stable across historical periods.

12. Evaluation Metrics and Formulas

Metric	Formula	Meaning	Project interpretation
MAE	$(1/n) \sum y - \hat{y} $	Average absolute error; same unit as visitors; lower is better.	94,563.52 test MAE means the average absolute miss is about 94.6 thousand visitors.
MSE	$(1/n) \sum (y - \hat{y})^2$	Squares errors; strongly penalizes large misses; lower is better.	17.0336 billion visitor ² for Linear Regression.
RMSE	$\sqrt{\text{MSE}}$	Returns to visitor units while retaining sensitivity to large errors; lower is better.	130,512.71 visitors for Linear Regression.
R ²	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Variance explained relative to a mean baseline; higher is better.	0.6539 means about 65.39% of test-target variance is explained.

12.1 MAE example

If errors are +100, -200 and +50 visitors, $\text{MAE} = (100+200+50)/3 = 116.67$ visitors.

12.2 MSE example

$\text{MSE} = (100^2 + 200^2 + 50^2)/3 = 17,500$ visitor². The large 200-error contributes 40,000 to the numerator, so large errors receive more weight.

12.3 RMSE example

$\text{RMSE} = \sqrt{17,500} \approx 132.29$ visitors. RMSE is easier to interpret than MSE because it is back in visitor units.

12.4 R² example

For actual values [100, 200, 300] and predictions [110, 190, 310], the residual sum of squares is 300 and total sum of squares is 20,000, giving $R^2 = 1 - 300/20,000 = 0.985$.

12.5 Is the model 65.39% accurate?

Viva correction

No. Regression does not normally use classification accuracy. $R^2 = 0.6539$ means the model explains about 65.39% of the variance in the held-out target relative to the mean baseline. It does not mean 65.39% of predictions are 'correct'.

12.6 What if a metric is not used?

- If MAE is omitted, the average practical error is less directly visible.
- If MSE is omitted, the extra penalty for very large errors is less visible.
- If RMSE is omitted, there is less direct same-unit sensitivity to large errors.
- If R² is omitted, we lose a normalized comparison against a mean baseline.
- Using several metrics gives a more complete picture because no single metric captures every aspect of forecasting error.

12.7 What is R1?

R1 is not a standard regression metric used by this project. If an examiner says 'R1', clarify whether they mean R^2 . The implemented code uses `r2_score`.

13. Model Comparison and Final Interpretation

13.1 Why compare models?

Different algorithms make different assumptions. Linear Regression tests whether engineered features explain demand approximately through a linear relationship. Polynomial Regression tests nonlinear time-trend curvature. Random Forest tests nonlinear threshold and interaction effects. Comparing them on the same chronological test set gives evidence rather than choosing an algorithm by name.

Model	MAE	MSE	RMSE	R2
Linear Regression	94,563.52	17,033,567,230.12	130,512.71	0.6539
Random Forest	184,914.52	48,371,647,698.26	219,935.55	0.0172
Polynomial Regression (Degree 2)	2,004,034.26	4,424,008,493,111.23	2,103,332.71	-88.8900
Polynomial Regression (Degree 4)	2,368,315.67	6,140,219,681,002.32	2,477,946.67	-123.76

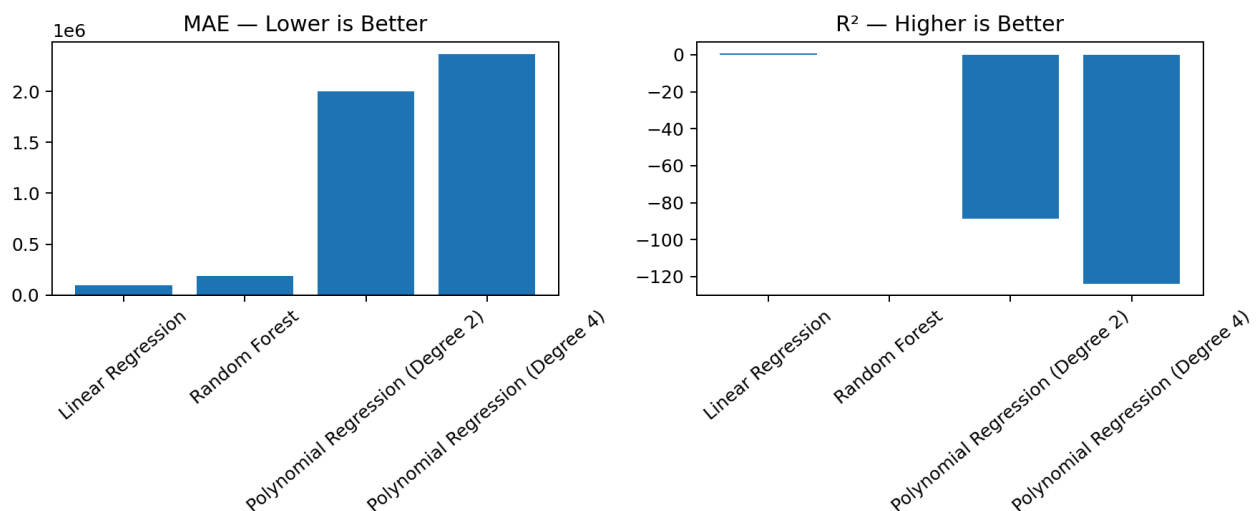


Figure 13. Test-set MAE and R^2 comparison using the exact generated results.

13.2 Test-set model comparison

The held-out test period is the latest 41 modeling months. Linear Regression has MAE 94,563.52, RMSE 130,512.71 and R^2 0.6539. Random Forest has MAE 184,914.52, RMSE 219,935.55 and R^2 0.0172. Polynomial degree 2 and degree 4 have very large errors and strongly negative R^2 .

13.3 Negative R^2

R^2 can be negative. It means the model is performing worse than the simple baseline that predicts the mean of the evaluation target. It does not mean the prediction itself is negative.

13.4 Actual vs predicted

The application plots actual and predicted values over the test period. The purpose is to see where the model tracks the series and where it misses, especially around sharp changes.

13.5 Random Forest feature importance

Feature	Importance
Lag_1	0.6188
Lag_12	0.1599
Year	0.1284
Lag_2	0.0385
Rolling Mean_3	0.0216
Rolling Mean_6	0.0118
Lag_3	0.0100
Rolling Mean_12	0.0060
Lag_6	0.0022
Month_Sin	0.0011
Month	0.0009
Month_Cos	0.0004
Quarter	0.0003

Lag_1 has the largest importance in the current fitted Random Forest, followed by Lag_12 and Year. Feature importance measures model reliance within the fitted forest; it does not prove causation.

13.6 Time-series CV results

Model	Fold	MAE	MSE	RMSE	R2
Linear Regression	1	76,236.10	10,090,381,480.41	100,450.89	-0.1155
Linear Regression	2	48,550.22	4,412,354,232.20	66,425.55	0.7630
Linear Regression	3	113,640.19	73,513,957,612.70	271,134.57	0.6313
Linear Regression	4	151,816.77	31,513,955,633.21	177,521.70	0.7656
Linear Regression	5	97,404.34	18,249,503,852.88	135,090.72	0.2286
Random Forest	1	110,422.52	18,817,440,477.65	137,176.68	-1.0803
Random Forest	2	96,771.86	13,994,127,043.44	118,296.78	0.2483
Random Forest	3	213,344.38	205,052,428,289.21	452,827.15	-0.0285
Random Forest	4	377,317.04	171,871,976,872.68	414,574.45	-0.2784
Random Forest	5	125,821.47	26,475,131,913.09	162,711.81	-0.1191

13.7 Cross-validation summary

Model	MAE mean	MAE std	RMSE mean	RMSE std	R ² mean	R ² std
Linear Regression	97,529.52	38,919.97	150,124.69	79,203.91	0.4546	0.3870
Random Forest	184,735.45	116,872.80	257,117.37	162,530.30	-0.2516	0.5012

The CV summary again shows Linear Regression with lower mean MAE/RMSE and higher mean R^2 than Random Forest in the current implementation.

13.8 How to finalize the result

The project does not hard-code a single final model. Both Linear Regression and Random Forest are fit as final models and exposed on the Forecast page. For the academic interpretation, Linear Regression is the primary model based on the supplied test and CV results, while Random Forest remains available as the nonlinear comparison model.

14. Forecasting Logic and Recursive Prediction



Figure 14. Recursive multi-step forecasting logic.

14.1 Why recursive forecasting?

For a one-step forecast, the model can use actual historical values. For month $t+2$, the actual $t+1$ value is not known yet. The project therefore predicts $t+1$, appends that prediction to history, and uses it as an input for the next step.

14.2 Exact forecast example

Month	Linear Regression Forecast
Feb 2026	1389780
Mar 2026	1396011
Apr 2026	1359678
May 2026	1312700
Jun 2026	1308844
Jul 2026	1507626

Displayed 6-month total in the application: approximately 8,274,640 visitors; average approximately 1,379,107 visitors/month.

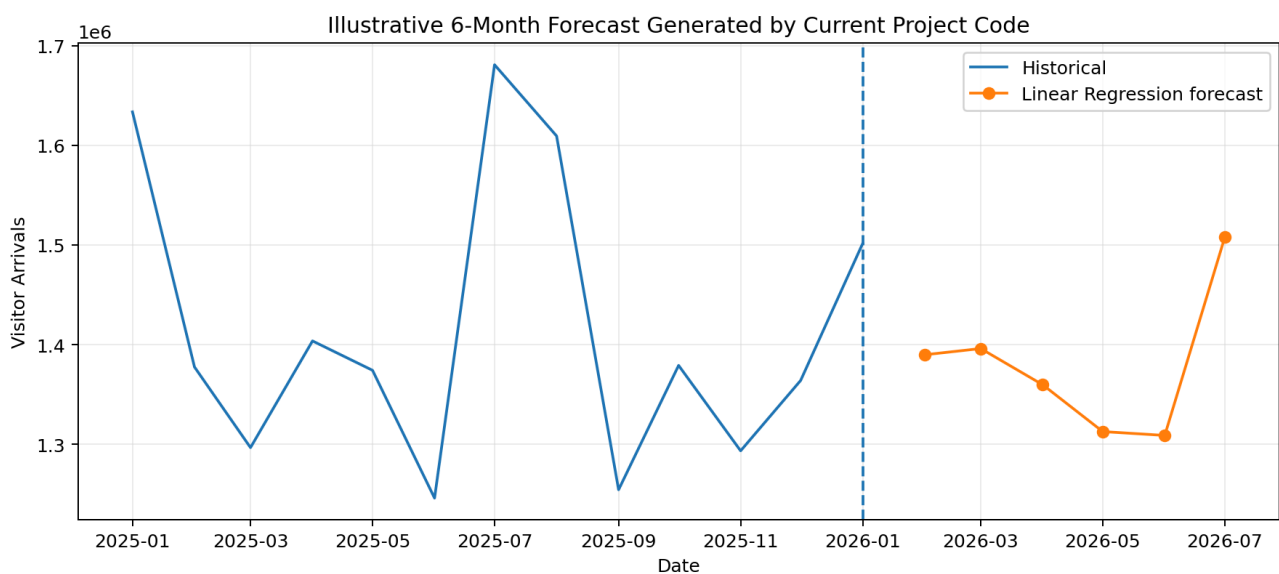


Figure 15. Six-month Linear Regression forecast using the exact current project code/results.

14.3 Specific-month selection

If the user asks for a later month such as October 2026, the application forecasts the intermediate months internally, then displays the selected target month. This is necessary because later lag features depend on earlier predicted values.

14.4 Forecast safeguards

The recursive_forecast function applies $\max(0.0, \text{prediction})$, preventing a negative visitor-count output.

14.5 Forecast uncertainty

The current app returns point forecasts, not prediction intervals. Therefore the values should be treated as model estimates, not guaranteed visitor counts.

15. Streamlit Application and User Flow

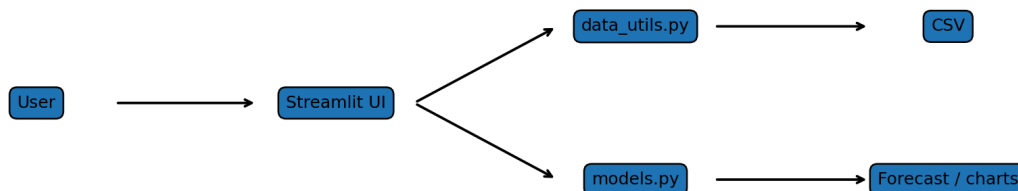


Figure 16. Streamlit application architecture.

15.1 Main pages

Page	Purpose
Home	Project summary, dataset information, quality checks, pipeline, historical demand and project mapping.
EDA	Dataset overview, statistics, distribution, outliers, trend, YoY, seasonality, heatmap, rolling means, lags, autocorrelation and feature relationships.
Model Comparison	Test metrics, actual vs predicted, Random Forest feature importance, TimeSeriesSplit results and CV summary.
Forecast	Select Linear Regression or Random Forest, choose next N months or specific months, generate recursive forecast and inspect chart/table.

15.2 Caching

The current app uses @st.cache_data for prepared data and @st.cache_resource for trained project models. This means the app can avoid recomputing the same expensive data/model work on every Streamlit rerun within the cache rules.

15.3 Why no .pkl file?

The project intentionally does not serialize model files. app.py imports the model-building functions, trains final models from the current feature data and caches them in memory. This keeps the project simple and avoids a separate model-artifact lifecycle.

15.4 Screenshot evidence

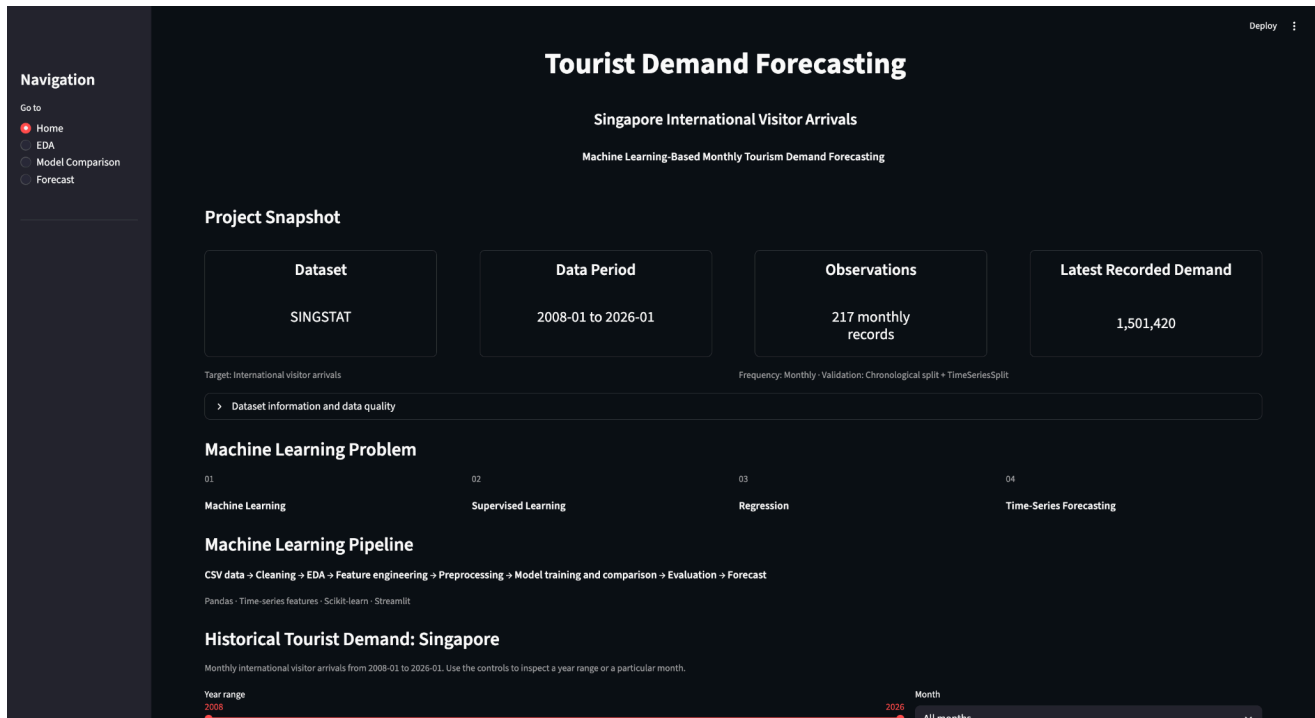


Figure 17. Home page — project snapshot and ML pipeline.

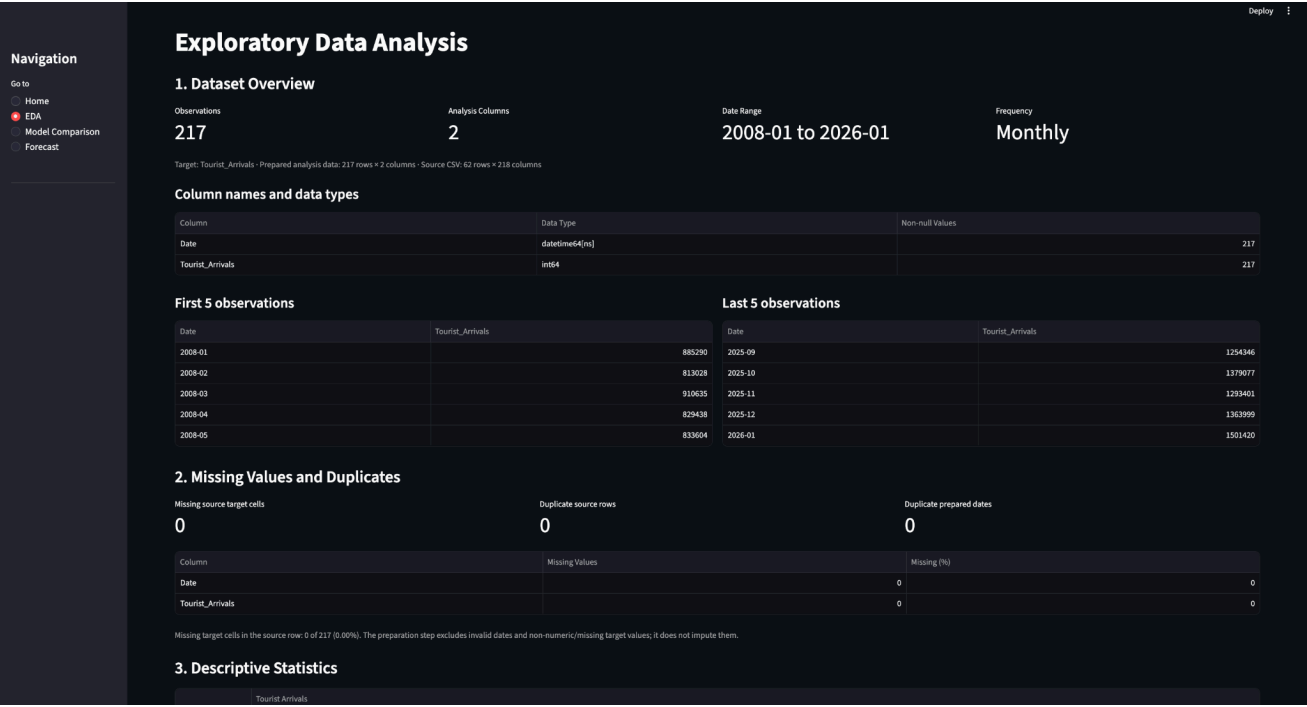




Figure 20. Year × Month seasonality heatmap.

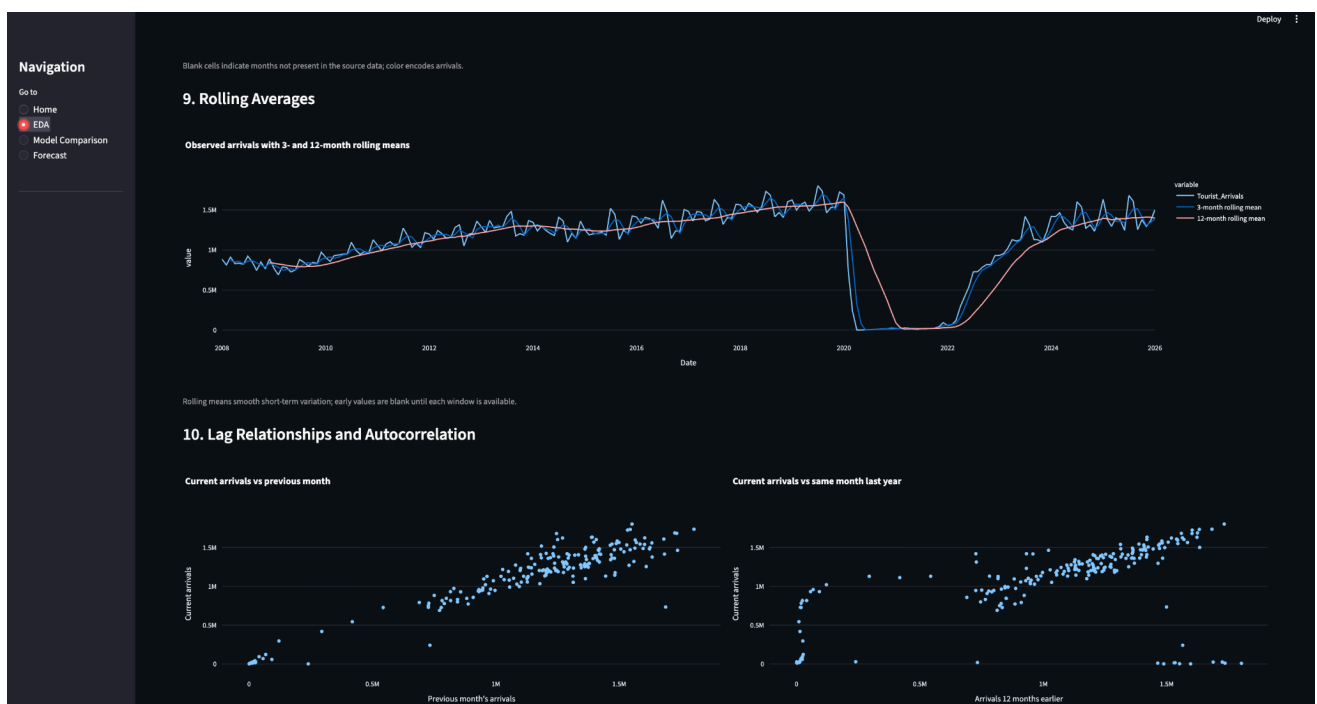


Figure 21. Rolling averages and lag relationships.

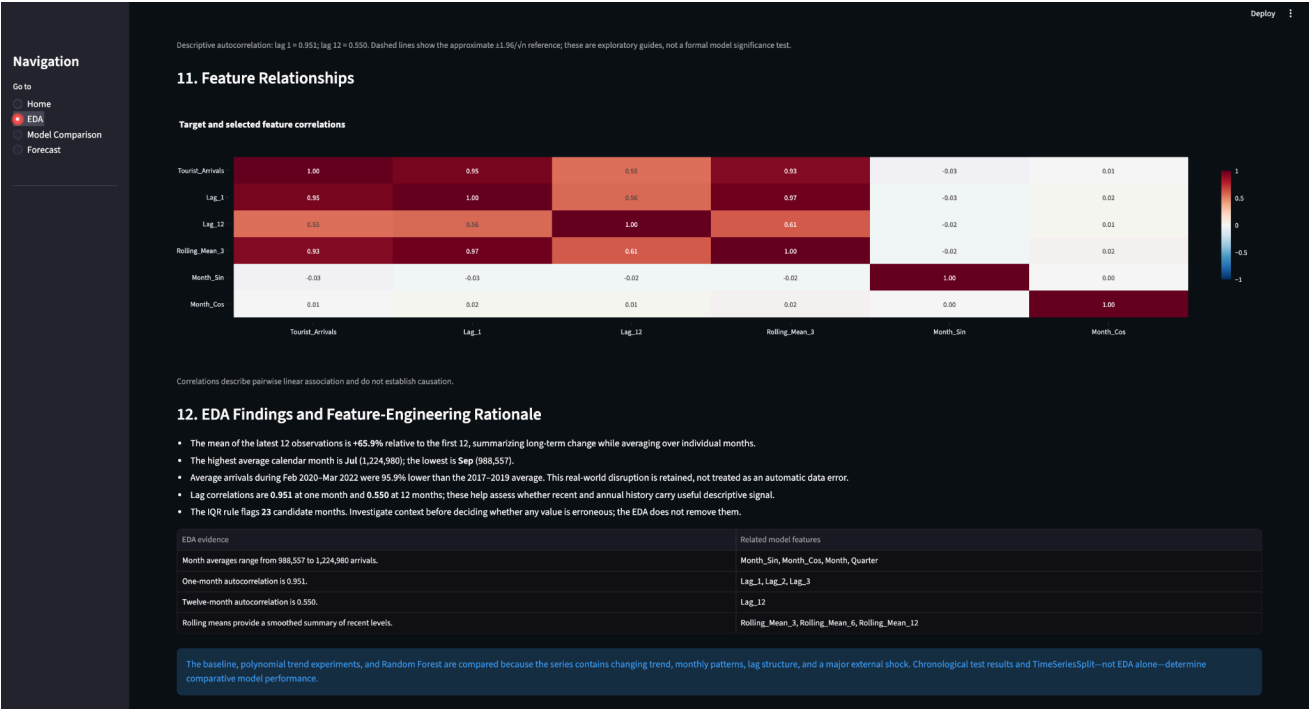


Figure 22. Feature relationships and EDA rationale.

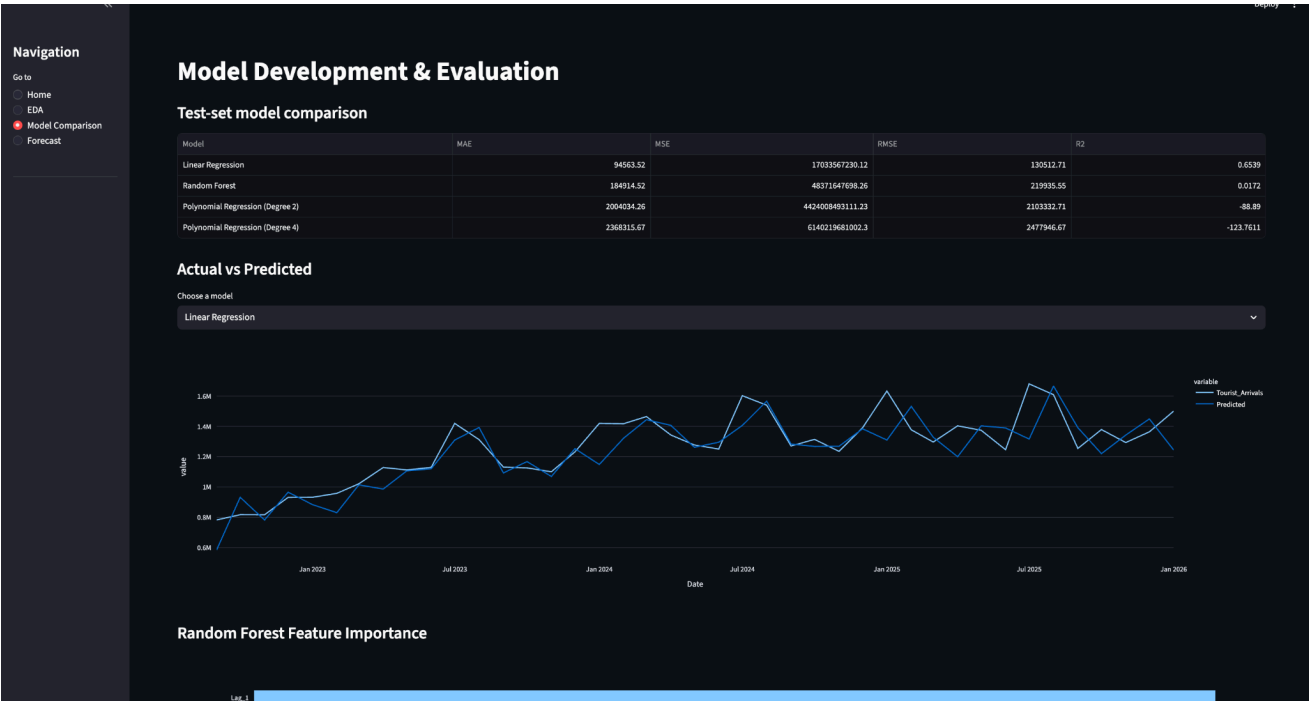


Figure 23. Model Development & Evaluation page.

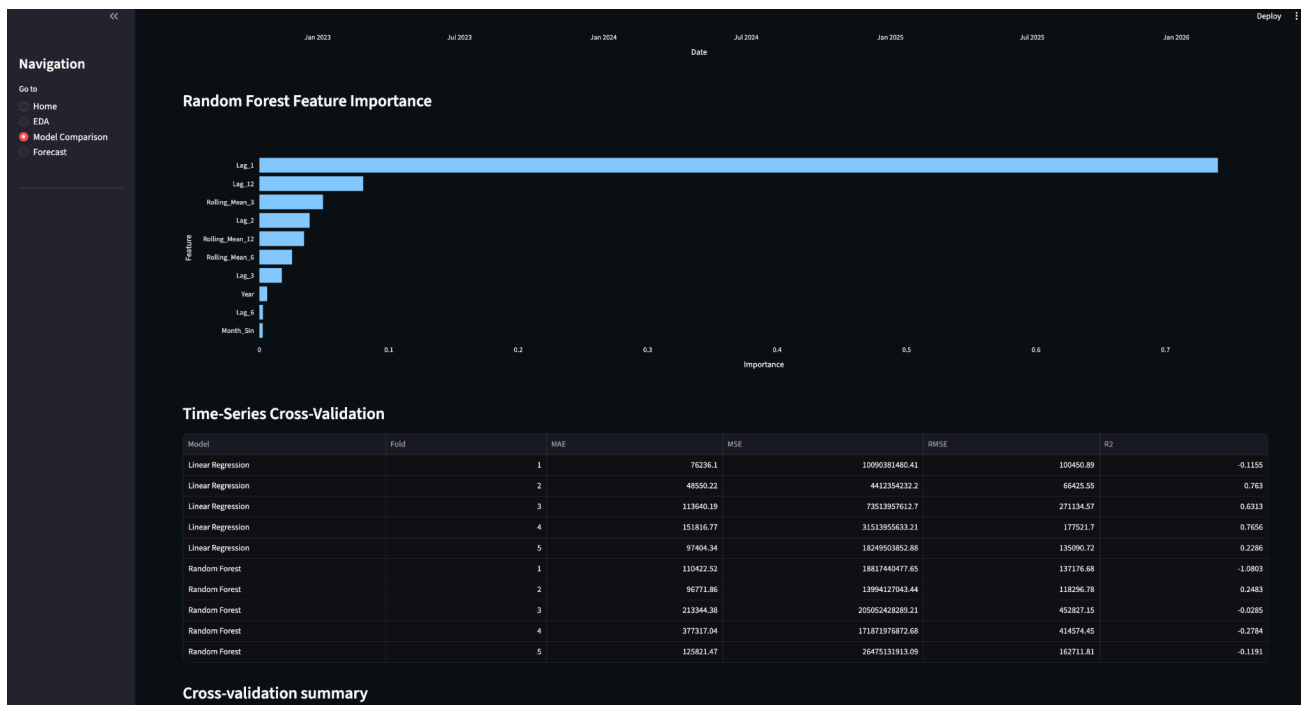


Figure 24. Random Forest feature importance and TimeSeriesSplit.

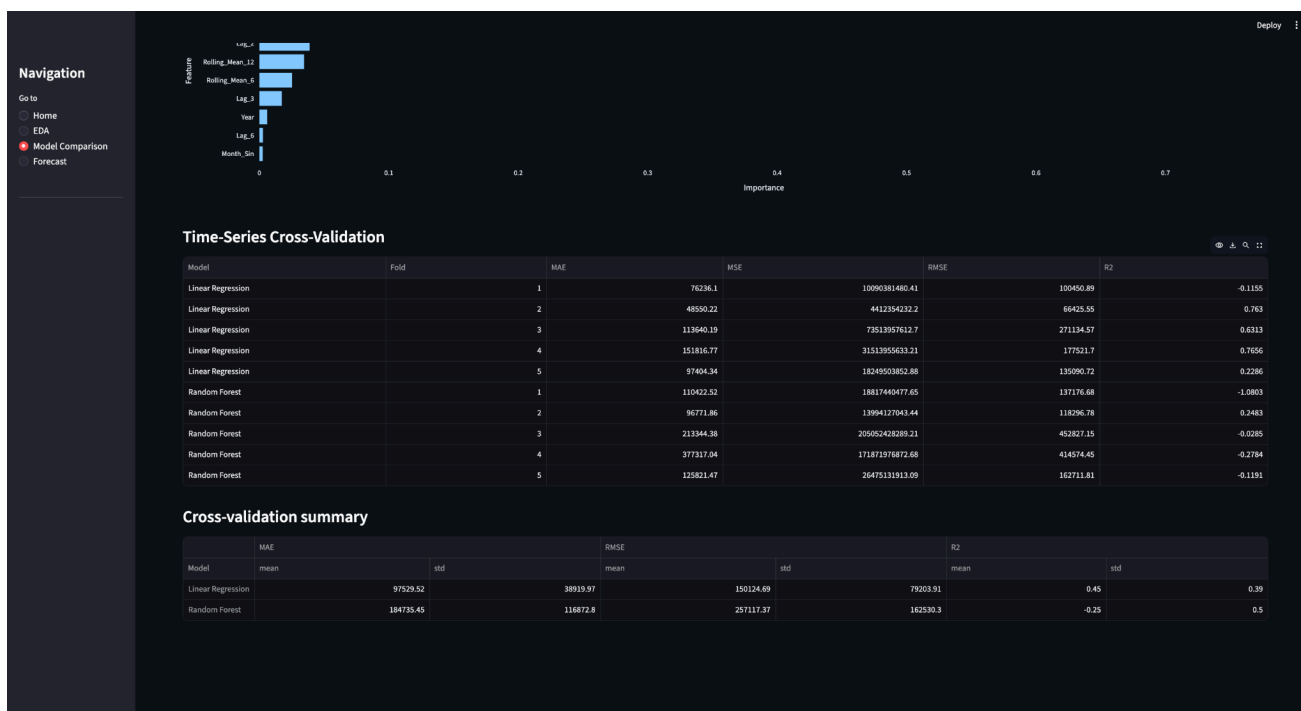


Figure 25. Time-series cross-validation and summary.

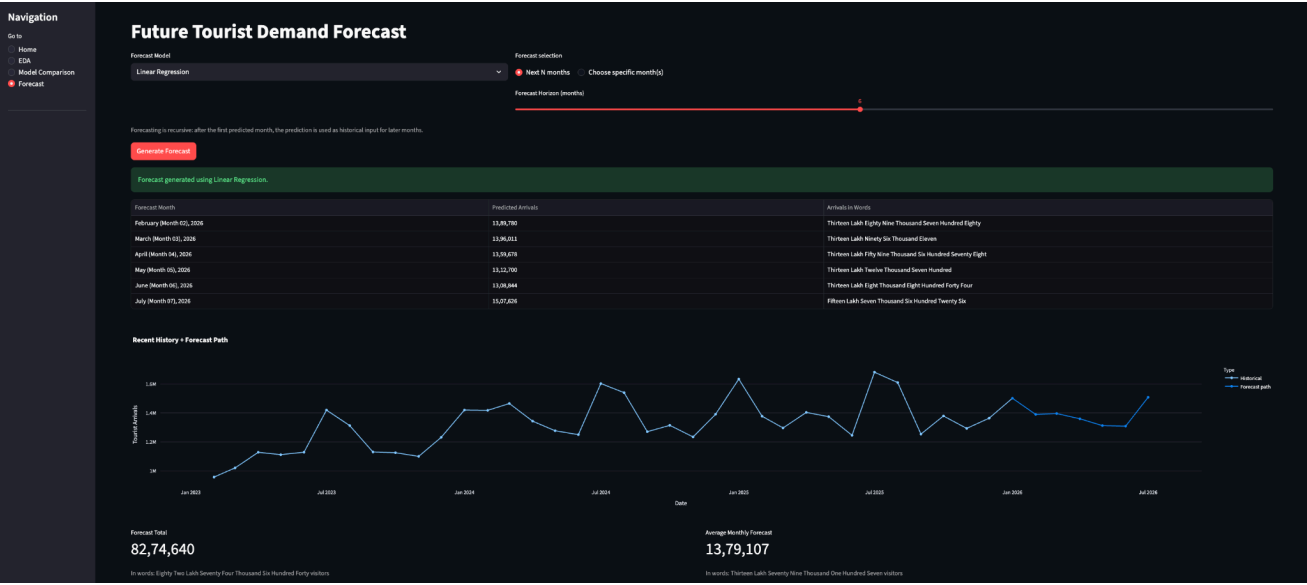


Figure 26. Forecast page with generated future-demand path.

16. Project Files and Code Explanation

The VS Code project screenshot shows the main project resources, including the data folder, source modules, model-result CSV files, README, requirements and viva material. Python `__pycache__` files are generated bytecode caches and are not the source code.

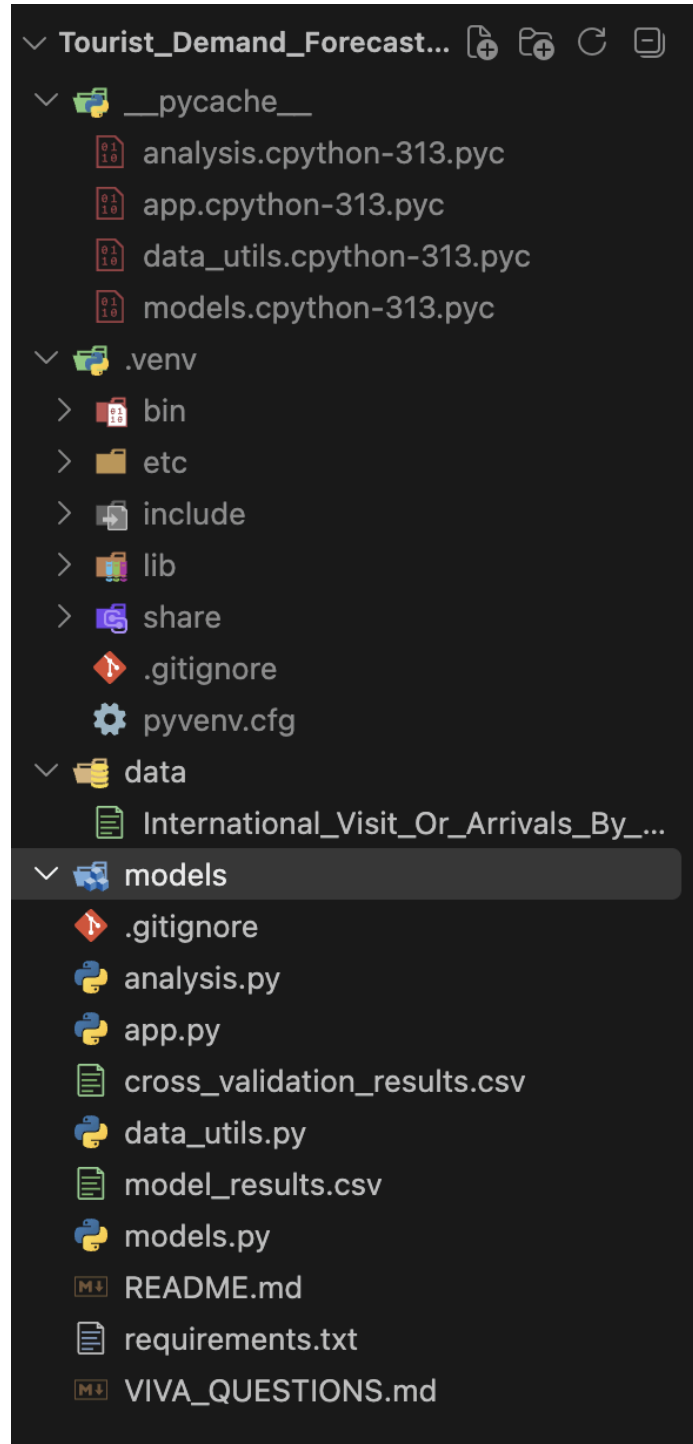


Figure 27. Supplied VS Code project-folder screenshot.

File / folder	Role
data/International_Visit_Or_Arrivals_By_Place_Of_Residence_Monthly.csv	Raw monthly tourism dataset.
data_utils.py	Data loading, target extraction, quality report, feature engineering and future feature construction.

models.py	Model pipelines, train/test split, evaluation metrics, model comparison, TimeSeriesSplit, feature importance and recursive forecast.
analysis.py	Command-line analysis driver that loads data, prints quality, model comparison, CV and recent predictions.
app.py	Streamlit UI, cached data/model training, EDA visualizations, model comparison and forecast interface.
model_results.csv	Generated test-set metrics for all four experiments.
cross_validation_results.csv	Generated fold-by-fold TimeSeriesSplit metrics.
README.md	Project setup, formulation, models, evaluation and run instructions.
FINAL_REPORT_DRAFT.md	Draft report structure and current results.
PRESENTATION_OUTLINE.md	Slide-by-slide presentation structure.
VIVA QUESTIONS.md	60-question viva checklist.
requirements.txt	Shown in the supplied VS Code screenshot; the exact contents were not present in the materialized project snapshot, so no dependency list is invented here.
.venv	Python virtual environment shown in the screenshot; environment files are not source logic.
pycache	Python-generated bytecode cache; not manually edited source.

16.1 data_utils.py – exact logic

Exact supplied data_utils.py source

```

1  import numpy as np
2  import pandas as pd
3
4  TARGET_ROW = "Total International Visitor Arrivals By Place Of Residence"
5
6  def load_raw_data(csv_path):
7      return pd.read_csv(csv_path)
8
9  def prepare_total_series(raw_df):
10     """Extract the total tourist-arrivals series and convert wide data to long time-series data."""
11     match = raw_df["DataSeries"].astype(str).str.strip().eq(TARGET_ROW)
12     if not match.any():
13         raise ValueError(f"Target series not found: {TARGET_ROW}")
14
15     row = raw_df.loc[match].iloc[0]
16     values = row.iloc[1:]
17     dates = pd.to_datetime(values.index, format="%Y-%m", errors="coerce")
18     arrivals = pd.to_numeric(values.values, errors="coerce")
19
20     df = pd.DataFrame({"Date": dates, "Tourist_Arrivals": arrivals})
21     df = df.dropna(subset=["Date", "Tourist_Arrivals"])
22     df = df.drop_duplicates(subset=["Date"]).sort_values("Date").reset_index(drop=True)
23     return df
24
25 def data_quality_report(raw_df, series_df):
26     return {
27         "raw_rows": int(raw_df.shape[0]),
28         "raw_columns": int(raw_df.shape[1]),
29         "raw_missing_cells": int(raw_df.isna().sum().sum()),
30         "raw_duplicate_rows": int(raw_df.duplicated().sum()),
31         "series_rows": int(series_df.shape[0]),
32         "series_missing_values": int(series_df["Tourist_Arrivals"].isna().sum()),
33         "series_duplicate_dates": int(series_df["Date"].duplicated().sum()),
34         "start_date": series_df["Date"].min().strftime("%Y-%m"),
35         "end_date": series_df["Date"].max().strftime("%Y-%m"),
36     }
37
38 def add_features(df):
39     out = df.copy()
40     out["Year"] = out["Date"].dt.year
41     out["Month"] = out["Date"].dt.month
42     out["Quarter"] = out["Date"].dt.quarter
43     out["Month_Sin"] = np.sin(2 * np.pi * out["Month"] / 12)
44     out["Month_Cos"] = np.cos(2 * np.pi * out["Month"] / 12)
45
46     for lag in [1, 2, 3, 6, 12]:
47         out[f"Lag_{lag}"] = out["Tourist_Arrivals"].shift(lag)
48
49     for window in [3, 6, 12]:
50         out[f"Rolling_Mean_{window}"] = (
51             out["Tourist_Arrivals"].shift(1).rolling(window).mean()
52         )
53     return out
54

```

```

55 def make_model_data(series_df):
56     featured = add_features(series_df).dropna().reset_index(drop=True)
57     features = [
58         "Year", "Month", "Quarter", "Month_Sin", "Month_Cos",
59         "Lag_1", "Lag_2", "Lag_3", "Lag_6", "Lag_12",
60         "Rolling_Mean_3", "Rolling_Mean_6", "Rolling_Mean_12"
61     ]
62     return featured, features
63
64 def make_future_features(history_df, next_date):
65     """Create one recursive forecast row using actual + previously predicted history."""
66     history = history_df.copy()
67     history["Date"] = pd.to_datetime(history["Date"])
68     history = history.sort_values("Date").reset_index(drop=True)
69
70     month = next_date.month
71     row = {
72         "Year": next_date.year,
73         "Month": month,
74         "Quarter": next_date.quarter,
75         "Month_Sin": np.sin(2 * np.pi * month / 12),
76         "Month_Cos": np.cos(2 * np.pi * month / 12),
77     }
78
79     values = history["Tourist_Arrivals"]
80     for lag in [1, 2, 3, 6, 12]:
81         row[f"Lag_{lag}"] = float(values.iloc[-lag])
82
83     for window in [3, 6, 12]:
84         row[f"Rolling_Mean_{window}"] = float(values.iloc[-window:].mean())
85
86     return pd.DataFrame([row])

```

16.2 data_utils.py line/function explanation

Code component	Meaning
Imports	numpy and pandas provide numerical arrays, date handling and tabular data.
TARGET_ROW	Stores the exact official total-series name.
load_raw_data	Reads the CSV into a DataFrame.
prepare_total_series	Finds the total row, treats monthly columns as dates, converts values to numeric and returns Date/Tourist_Arrivals.
data_quality_report	Counts rows, columns, missing cells, duplicates and date range.
add_features	Adds calendar, cyclical, lag and rolling features.
make_model_data	Drops rows without complete features and returns the modeling feature list.
make_future_features	Builds one future feature row from actual + previously predicted history.

16.3 models.py — exact logic

Exact supplied models.py source

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.linear_model import LinearRegression
4 from sklearn.preprocessing import OneHotEncoder, StandardScaler, PolynomialFeatures
5 from sklearn.compose import ColumnTransformer
6 from sklearn.pipeline import Pipeline
7 from sklearn.ensemble import RandomForestRegressor
8 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
9 from sklearn.model_selection import TimeSeriesSplit
10
11 from data_utils import add_features, make_model_data, make_future_features
12
13 FEATURES = [
14     "Year", "Month", "Quarter", "Month_Sin", "Month_Cos",
15     "Lag_1", "Lag_2", "Lag_3", "Lag_6", "Lag_12",
16     "Rolling_Mean_3", "Rolling_Mean_6", "Rolling_Mean_12"
17 ]
18 CATEGORICAL = ["Month", "Quarter"]
19 NUMERIC = [x for x in FEATURES if x not in CATEGORICAL]
20
21 def build_linear_model():
22     preprocessor = ColumnTransformer([
23         ("num", StandardScaler(), NUMERIC),
24         ("cat", OneHotEncoder(handle_unknown="ignore"), CATEGORICAL),
25     ])
26     return Pipeline([
27         ("preprocessor", preprocessor),
28         ("model", LinearRegression())
29     ])
30
31 def build_random_forest():
32     return RandomForestRegressor(
33         n_estimators=300,
34         max_depth=10,
35         min_samples_leaf=2,
36         random_state=42,
37         n_jobs=-1
38     )
39
40 def build_polynomial_model(degree):
41     return Pipeline([
42         ("poly", PolynomialFeatures(degree=degree, include_bias=False)),
43         ("scaler", StandardScaler()),
44         ("model", LinearRegression())
45     ])
46
47 def chronological_split(featured_df, test_size=0.20):
48     split = int(len(featured_df) * (1 - test_size))
49     return featured_df.iloc[:split].copy(), featured_df.iloc[split:].copy()
50
51 def evaluate(y_true, y_pred):
52     mse = mean_squared_error(y_true, y_pred)
53     return {
54         "MAE": mean_absolute_error(y_true, y_pred),
55         "MSE": mse,
56         "RMSE": np.sqrt(mse),
57         "R2": r2_score(y_true, y_pred)
58     }
59
60 def train_and_compare(featured_df):
61     train_df, test_df = chronological_split(featured_df)
62     X_train, y_train = train_df[FEATURES], train_df["Tourist_Arrivals"]
63     X_test, y_test = test_df[FEATURES], test_df["Tourist_Arrivals"]
64
65     models = {
66         "Linear Regression": build_linear_model(),
67         "Random Forest": build_random_forest(),
68     }
69
70     rows = []
71     fitted = {}
72     predictions = {}
73
74     for name, model in models.items():
75         model.fit(X_train, y_train)
76         pred = model.predict(X_test)
77         metrics = evaluate(y_test, pred)
78         rows.append({"Model": name, **metrics})
79         fitted[name] = model
80         predictions[name] = pred
81
82     # Polynomial regression is deliberately a simple time-trend experiment,
```

```

83     # matching the syllabus's degree-2 vs degree-4 curve-fitting activity.
84     time_train = np.arange(len(train_df)).reshape(-1, 1)
85     time_test = np.arange(len(train_df), len(featured_df)).reshape(-1, 1)
86
87     for degree in [2, 4]:
88         model = build_polynomial_model(degree)
89         model.fit(time_train, y_train)
90         pred = model.predict(time_test)
91         metrics = evaluate(y_test, pred)
92         rows.append({"Model": f"Polynomial Regression (Degree {degree})", **metrics})
93         fitted[f"Polynomial Regression (Degree {degree})"] = model
94         predictions[f"Polynomial Regression (Degree {degree})"] = pred
95
96     results = pd.DataFrame(rows)
97     actual_vs_pred = test_df[["Date", "Tourist_Arrivals"]].copy()
98     for name, pred in predictions.items():
99         actual_vs_pred[name] = pred
100
101     return results, fitted, actual_vs_pred, train_df, test_df
102
103 def time_series_cv(featured_df, n_splits=5):
104     X = featured_df[FEATURES]
105     y = featured_df["Tourist_Arrivals"]
106     tscv = TimeSeriesSplit(n_splits=n_splits)
107     records = []
108
109     for model_name in ["Linear Regression", "Random Forest"]:
110         for fold, (train_idx, test_idx) in enumerate(tscv.split(X), 1):
111             model = build_linear_model() if model_name == "Linear Regression" else build_random_forest()
112             model.fit(X.iloc[train_idx], y.iloc[train_idx])
113             pred = model.predict(X.iloc[test_idx])
114             m = evaluate(y.iloc[test_idx], pred)
115             records.append({"Model": model_name, "Fold": fold, **m})
116
117     return pd.DataFrame(records)
118
119 def random_forest_feature_importance(model, feature_names=FEATURES):
120     """Return Random Forest feature importance values."""
121     if not hasattr(model, "feature_importances_"):
122         raise ValueError("The supplied model is not a fitted Random Forest.")
123     return pd.DataFrame({
124         "Feature": feature_names,
125         "Importance": model.feature_importances_
126     }).sort_values("Importance", ascending=False).reset_index(drop=True)
127
128 def fit_final_models(featured_df):
129     X = featured_df[FEATURES]
130     y = featured_df["Tourist_Arrivals"]
131     models = {
132         "Linear Regression": build_linear_model(),
133         "Random Forest": build_random_forest(),
134     }
135     for model in models.values():
136         model.fit(X, y)
137     return models
138
139 def recursive_forecast(model, history_df, months=12):
140     history = history_df[["Date", "Tourist_Arrivals"]].copy()
141     history["Date"] = pd.to_datetime(history["Date"])
142     history = history.sort_values("Date").reset_index(drop=True)
143
144     forecasts = []
145     last_date = history["Date"].iloc[-1]
146
147     for _ in range(months):
148         next_date = last_date + pd.DateOffset(months=1)
149         X_next = make_future_features(history, next_date)
150         prediction = float(model.predict(X_next[FEATURES])[0])
151         prediction = max(0.0, prediction)
152
153         forecasts.append({
154             "Date": next_date,
155             "Predicted_Tourist_Arrivals": prediction
156         })
157
158     history = pd.concat([
159         history,
160         pd.DataFrame({"Date": [next_date], "Tourist_Arrivals": [prediction]})
161     ], ignore_index=True)
162     last_date = next_date
163
164     return pd.DataFrame(forecasts)

```

16.4 models.py line/function explanation

Function/section	Meaning
FEATURES	The 13 model inputs.
CATEGORICAL / NUMERIC	Separates Month/Quarter from numeric features for the Linear pipeline.
build_linear_model	Standardizes numeric features, one-hot encodes Month/Quarter, then fits LinearRegression.
build_random_forest	Creates the 300-tree RandomForestRegressor with the supplied hyperparameters.
build_polynomial_model	Creates PolynomialFeatures → StandardScaler → LinearRegression.
chronological_split	Takes the first 80% for training and latest 20% for testing.
evaluate	Calculates MAE, MSE, RMSE and R ² .
train_and_compare	Fits Linear/RF, then degree-2/4 trend experiments, and returns results plus predictions.
time_series_cv	Runs five sequential folds for Linear and Random Forest.
random_forest_feature_importance	Extracts and sorts feature importances .
fit_final_models	Fits Linear and Random Forest on all modeling rows for forecasting.
recursive_forecast	Predicts month-by-month and appends each prediction to history.

16.5 analysis.py — exact logic

Exact supplied analysis.py source

```
1 import os
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.cluster import KMeans
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.decomposition import PCA
7
8 from data_utils import load_raw_data, prepare_total_series, data_quality_report, make_model_data
9 from models import train_and_compare, time_series_cv
10
11 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
12 CSV_PATH = os.path.join(BASE_DIR, "data",
13 "International_Visit_Or_Arrivals_By_Place_Of_Residence_Monthly.csv")
14
15 def main():
16     raw = load_raw_data(CSV_PATH)
17     series = prepare_total_series(raw)
18     quality = data_quality_report(raw, series)
19
20     print("DATA QUALITY REPORT")
21     for k, v in quality.items():
22         print(f"{k}: {v}")
23
24     featured, _ = make_model_data(series)
25     results, _, actual_vs_pred, train_df, test_df = train_and_compare(featured)
26     print("\nMODEL COMPARISON")
27     print(results.round(2).to_string(index=False))
28
29     cv = time_series_cv(featured)
30     print("\nTIME-SERIES CROSS-VALIDATION")
31     print(cv.round(2).to_string(index=False))
32
33     print("\nTEST SET PREDICTIONS")
34     print(actual_vs_pred.tail(10).round(2).to_string(index=False))
35
36     print("\nData saved/available through:", series["Date"].max().strftime("%Y-%m"))
37
38 if __name__ == "__main__":
39     main()
```

16.6 analysis.py explanation

- Imports the data utilities and model functions.
- Builds CSV_PATH relative to the script location.
- Loads raw data and prepares the total series.
- Prints the quality report.
- Creates modeling features.
- Runs model comparison and prints rounded results.
- Runs five-fold time-series cross-validation and prints results.
- Prints the last ten test-set prediction rows.
- Prints the latest available source date.

Code-quality note

analysis.py imports KMeans, StandardScaler, PCA and matplotlib, but the current main() does not use them. They are not part of the implemented forecasting pipeline. The project should not claim that clustering or PCA is used unless those imports are later activated in code.

16.7 app.py — complete source listing

The supplied app.py contains 819 source lines. The complete source is preserved below in compact chunks. The large CSS/HTML formatting sections are application presentation code; the functional ML logic is described in the preceding sections.

Exact supplied app.py — source lines 1–120

```
1 import os
2 import sys
3 import pandas as pd
4 import numpy as np
5 import streamlit as st
6 import plotly.express as px
7
8 # Make local modules importable when Streamlit runs from the project root.
9 sys.path.append(os.path.dirname(os.path.abspath(__file__)))
10
11 from data_utils import load_raw_data, prepare_total_series, data_quality_report, make_model_data
12 from models import train_and_compare, time_series_cv, fit_final_models, recursive_forecast, random_forest_feature_importance
13
14 st.set_page_config(
15     page_title="Singapore Tourist Demand Forecasting",
16     layout="wide"
17 )
18
19 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
20 CSV_PATH = os.path.join(
21     BASE_DIR, "data",
22     "International_Visit_Or_Arrivals_By_Place_Of_Residence_Monthly.csv"
23 )
24
25 @st.cache_data
26 def load_project_data():
27     raw = load_raw_data(CSV_PATH)
28     series = prepare_total_series(raw)
29     quality = data_quality_report(raw, series)
30     featured, _ = make_model_data(series)
31     return raw, series, featured, quality
32
33 @st.cache_resource
34 def train_project_models(featured):
35     results, _, actual_vs_pred, train_df, test_df = train_and_compare(featured)
36     final_models = fit_final_models(featured)
37     cv = time_series_cv(featured)
38     return results, actual_vs_pred, train_df, test_df, final_models, cv
39
40 def format_indian_number(value):
41     number = int(round(float(value)))
42     sign = "-" if number < 0 else ""
43     digits = str(abs(number))
44     if len(digits) <= 3:
45         return sign + digits
46     trailing = digits[-3:]
47     leading = digits[:-3]
48     groups = []
49     while leading:
50         groups.append(leading[-2:])
51         leading = leading[:-2]
52     return sign + ",".join([*reversed(groups), trailing])
53
54 def number_to_indian_words(value):
55     ones = [
56         "Zero", "One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine",
57         "Ten", "Eleven", "Twelve", "Thirteen", "Fourteen", "Fifteen", "Sixteen",
58         "Seventeen", "Eighteen", "Nineteen",
59     ]
60     tens = ["", "", "Twenty", "Thirty", "Forty", "Fifty", "Sixty", "Seventy", "Eighty", "Ninety"]
61
62     def under_thousand(number):
63         parts = []
64         hundreds, remainder = divmod(number, 100)
65         if hundreds:
66             parts.extend([ones[hundreds], "Hundred"])
67         if remainder >= 20:
68             parts.append(tens[remainder // 10])
69             if remainder % 10:
70                 parts.append(ones[remainder % 10])
71         elif remainder:
72             parts.append(ones[remainder])
73         return " ".join(parts)
74
75     def convert(number):
76         for divisor, scale in [(10_000_000, "Crore"), (100_000, "Lakh"), (1_000, "Thousand")]:
77             if number >= divisor:
78                 quotient, remainder = divmod(number, divisor)
79                 return f"{convert(quotient)} {scale}" + (f" {convert(remainder)}" if remainder else "")
```

```

80         return under_thousand(number)
81
82     number = int(round(float(value)))
83     if number == 0:
84         return "Zero"
85     return ("Minus " if number < 0 else "") + convert(abs(number))
86
87 raw, series, featured, quality = load_project_data()
88 results, actual_vs_pred, train_df, test_df, final_models, cv = train_project_models(featured)
89
90 st.sidebar.title("Navigation")
91 page_options = ["Home", "EDA", "Model Comparison", "Forecast"]
92 page = st.sidebar.radio(
93     "Go to",
94     page_options,
95     key="page_selection"
96 )
97
98 def navigate_to(page_name):
99     st.session_state["page_selection"] = page_name
100
101 st.sidebar.markdown("---")
102
103 if page == "Home":
104     st.markdown(
105         """
106         <style>
107         .home-hero {
108             text-align: center;
109             padding: 0.75rem 0 3rem;
110         }
111         .home-hero h1 {
112             font-size: 2.75rem;
113             line-height: 1.15;
114             margin: 0 0 0.65rem;
115         }
116         .home-hero h2 {
117             font-size: 1.5rem;
118             line-height: 1.3;
119             margin: 0 0 0.65rem;
120         }

```

Exact supplied app.py — source lines 121–240

```

1         .home-hero p {
2             font-size: 1.05rem;
3             font-weight: 600;
4             margin: 0;
5         }
6     </style>
7     <div class="home-hero">
8         <h1>Tourist Demand Forecasting</h1>
9         <h2>Singapore International Visitor Arrivals</h2>
10        <p>Machine Learning-Based Monthly Tourism Demand Forecasting</p>
11    </div>
12    """,
13    unsafe_allow_html=True,
14 )
15
16 st.markdown("### Project Snapshot")
17 st.markdown(
18     """
19     <style>
20     [data-testid="stMetricLabel"] {
21         display: flex;
22         align-items: flex-start;
23         justify-content: center;
24         min-height: 3.6rem;
25         text-align: center;
26     }
27     [data-testid="stMetricLabel"] [data-testid="stMarkdownContainer"] p {
28         font-size: 1.5rem !important;
29         font-weight: 600;
30         line-height: 1.2;
31         margin: 0 !important;
32         text-align: center;
33         white-space: normal !important;
34         overflow: visible !important;
35         text-overflow: clip !important;
36     }
37     [data-testid="stMetricValue"] {
38         font-size: 1.4rem;
39         text-align: center;
40         white-space: normal;
41         overflow: visible;
42         text-overflow: clip;
43     }
44     [data-testid="stMetricValue"] [data-testid="stMarkdownContainer"] p {
45         font-size: 1.4rem !important;
46         line-height: 1.15;
47         margin-top: 0.9rem !important;
48         text-align: center;

```

```

49         white-space: normal !important;
50         overflow: visible !important;
51         text-overflow: clip !important;
52     }
53     [data-testid="stHorizontalBlock"] > [data-testid="stColumn"]:nth-child(4) [data-testid="stMetricValue"] {
54         padding-top: 1.5rem !important;
55     }
56     [data-testid="stHorizontalBlock"] > [data-testid="stColumn"]:nth-child(4) [data-testid="stMetricValue"]
[data-testid="stMarkdownContainer"] p {
57         margin-top: 0 !important;
58     }
59     </style>
60     """,
61     unsafe_allow_html=True,
62 )
63 c1, c2, c3, c4 = st.columns(4, gap="large", vertical_alignment="top")
64 c1.metric("Dataset", "SINGSTAT", border=True, height=160)
65 c2.metric("Data Period", f"{quality['start_date']} to {quality['end_date']}", border=True, height=160)
66 c3.metric("Observations", f"{quality['series_rows']} monthly \nrecords", border=True, height=160)
67 c4.metric("Latest Recorded Demand", f"{int(series['Tourist_Arrivals'].iloc[-1]):,}", border=True, height=160)
68
69 details_col1, details_col2 = st.columns(2)
70 details_col1.caption("Target: International visitor arrivals")
71 details_col2.caption("Frequency: Monthly · Validation: Chronological split + TimeSeriesSplit")
72
73 with st.expander("Dataset information and data quality"):
74     source_col, agency_col, frequency_col = st.columns(3)
75     source_col.markdown("***Dataset**\n\nInternational Visitor Arrivals by Place of Residence, Monthly")
76     agency_col.markdown("***Publisher**\n\nSingapore Department of Statistics (SINGSTAT)\n\nOriginal source: Singapore Tourism
Board")
77     frequency_col.markdown("***Series**\n\nMonthly\n\nTarget: Tourist arrivals")
78
79     quality_checks = [
80         ("Missing target values", quality["series_missing_values"] == 0,
81          f"{quality['series_missing_values']} in the prepared series"),
82         ("Duplicate dates", quality["series_duplicate_dates"] == 0,
83          f"{quality['series_duplicate_dates']} in the prepared series"),
84         ("Date format", pd.api.types.is_datetime64_any_dtype(series["Date"]),
85          "Converted to datetime"),
86         ("Numeric target", pd.api.types.is_numeric_dtype(series["Tourist_Arrivals"]),
87          "Arrivals converted to numeric values"),
88         ("Chronological order", series["Date"].is_monotonic_increasing,
89          "Dates sorted from oldest to newest"),
90     ]
91     checks_passed = all(check[1] for check in quality_checks)
92     if checks_passed:
93         st.success("Data quality status: Ready for modeling")
94     else:
95         st.warning("Data quality checks need attention.")
96     quality_table = pd.DataFrame([
97         {"Check": name, "Status": "Passed" if passed else "Review", "Details": detail}
98         for name, passed, detail in quality_checks
99     ])
100     st.dataframe(quality_table, use_container_width=True, hide_index=True)
101     st.caption(
102         f"Raw file checks: {quality['raw_missing_cells']} missing cells and "
103         f"{quality['raw_duplicate_rows']} duplicate rows."
104     )
105
106 st.markdown("### Machine Learning Problem")
107 problem_steps = st.columns(4)
108 for column, number, label in zip(
109     problem_steps,
110     ["01", "02", "03", "04"],
111     ["Machine Learning", "Supervised Learning", "Regression", "Time-Series Forecasting"]
112 ):
113     column.caption(number)
114     column.markdown(f"***{label}**")
115
116 st.markdown("### Machine Learning Pipeline")
117 st.markdown(
118     "**CSV data** → **Cleaning** → **EDA** → **Feature engineering** → "
119     "**Preprocessing** → **Model training and comparison** → **Evaluation** → **Forecast**"
120 )

```

Exact supplied app.py — source lines 241–360

```

1     st.caption("Pandas · Time-series features · Scikit-learn · Streamlit")
2
3     st.markdown("### Historical Tourist Demand: Singapore")
4     st.caption(
5         f"Monthly international visitor arrivals from {quality['start_date']} to {quality['end_date']}. "
6         "Use the controls to inspect a year range or a particular month."
7     )
8     year_options = list(range(2008, 2027))
9     filter_years, filter_month = st.columns([2, 1])
10    selected_years = filter_years.select_slider(
11        "Year range", options=year_options, value=(2008, 2026)
12    )
13    month_options = ["All months"] + list(
14        pd.date_range("2020-01-01", periods=12, freq="MS").strftime("%B")
15    )

```

```

16     selected_month = filter_month.selectbox("Month", month_options)
17
18     chart_series = series[
19         series["Date"].dt.year.between(selected_years[0], selected_years[1])
20     ]
21     if selected_month != "All months":
22         month_number = month_options.index(selected_month)
23         chart_series = chart_series[chart_series["Date"].dt.month == month_number]
24
25     if chart_series.empty:
26         st.info("No visitor-arrival records are available for this selection.")
27     else:
28         fig = px.line(
29             chart_series, x="Date", y="Tourist_Arrivals",
30             title="Historical Tourist Demand"
31         )
32         covid_start = pd.Timestamp("2020-02-01")
33         covid_end = pd.Timestamp("2022-03-31")
34         if chart_series["Date"].min() <= covid_end and chart_series["Date"].max() >= covid_start:
35             fig.add_vrect(
36                 x0=covid_start,
37                 x1=covid_end,
38                 fillcolor="#EF4444",
39                 opacity=0.14,
40                 line_width=0,
41                 annotation_text="COVID-19 impact",
42                 annotation_position="top left",
43                 annotation_font=dict(color="#F87171", size=12),
44             )
45             fig.update_layout(xaxis_title="Month", yaxis_title="Tourist Arrivals")
46             st.plotly_chart(fig, use_container_width=True)
47
48     st.markdown("### Historical Disruption")
49     st.info(
50         "Visitor arrivals fell exceptionally during the COVID-19 period, then recovered. "
51         "This disruption is important context when interpreting model errors and forecasts."
52     )
53
54     st.markdown("### Models Compared")
55     model_columns = st.columns(4)
56     model_details = [
57         ("Linear Regression", "Baseline", "Interpretable linear model."),
58         ("Polynomial Degree 2", "Non-linear experiment", "Fits a curved time trend."),
59         ("Polynomial Degree 4", "Non-linear experiment", "Tests a more flexible time trend."),
60         ("Random Forest", "Ensemble", "Combines decision trees and supports feature importance."),
61     ]
62     for column, (model_name, role, description) in zip(model_columns, model_details):
63         column.markdown(f"***{model_name}***")
64         column.caption(role)
65         column.write(description)
66
67     st.markdown("### Evaluation")
68     metric_columns = st.columns(4)
69     for column, metric_name, explanation in zip(
70         metric_columns,
71         ["MAE", "MSE", "RMSE", "R²"],
72         ["Mean absolute error", "Mean squared error", "Root mean squared error", "Explained variation"],
73     ):
74         column.markdown(f"***{metric_name}***")
75         column.caption(explanation)
76     st.caption("Lower MAE, MSE, and RMSE indicate smaller errors. Higher R² indicates more explained variation. TimeSeriesSplit is used for cross-validation.")
77
78     st.markdown("### Potential Applications")
79     st.write(
80         "Tourism demand estimates may help inform hotel capacity, transport, workforce, "
81         "marketing, and resource-planning decisions."
82     )
83
84     st.markdown("### Quick Access")
85     access_columns = st.columns(3)
86     quick_links = [
87         ("Explore EDA", "EDA"),
88         ("Compare Models", "Model Comparison"),
89         ("Forecast", "Forecast"),
90     ]
91     for column, (label, target_page) in zip(access_columns, quick_links):
92         column.button(
93             label,
94             on_click=navigate_to,
95             args=(target_page,),
96             use_container_width=True
97         )
98
99     elif page == "EDA":
100         st.title("Exploratory Data Analysis")
101
102         target_name = "Tourist_Arrivals"
103         target_row = raw.loc[
104             raw["DataSeries"].astype(str).str.strip()
105             == "Total International Visitor Arrivals By Place Of Residence"
106         ]
107         source_target_values = target_row.iloc[0, 1:] if not target_row.empty else pd.Series(dtype=float)
108         source_target_numeric = pd.to_numeric(source_target_values, errors="coerce")

```

```

109     source_target_missing = int(source_target_numeric.isna().sum())
110     source_target_missing_pct = (
111         source_target_missing / len(source_target_numeric) * 100
112         if len(source_target_numeric) else 0.0
113     )
114
115     series_for_eda = series.copy()
116     series_for_eda["Year"] = series_for_eda["Date"].dt.year
117     series_for_eda["Month"] = series_for_eda["Date"].dt.month
118     month_names = list(pd.date_range("2020-01-01", periods=12, freq="MS").strftime("%b"))
119     series_for_eda["Month_Name"] = pd.Categorical(
120         series_for_eda["Date"].dt.strftime("%b"), categories=month_names, ordered=True

```

Exact supplied app.py — source lines 361–480

```

1     )
2     display_series = series[["Date", target_name]].copy()
3     display_series["Date"] = display_series["Date"].dt.strftime("%Y-%m")
4
5     st.markdown("### 1. Dataset Overview")
6     overview_columns = st.columns(4)
7     overview_columns[0].metric("Observations", f"{len(series):,}")
8     overview_columns[1].metric("Analysis Columns", f"{len(series.columns)}")
9     overview_columns[2].metric("Date Range", f"{series['Date'].min():%Y-%m} to {series['Date'].max():%Y-%m}")
10    overview_columns[3].metric("Frequency", "Monthly")
11    st.caption(
12        f"Target: {target_name} · Prepared analysis data: {series.shape[0]:,} rows × "
13        f"{series.shape[1]} columns · Source CSV: {raw.shape[0]:,} rows × {raw.shape[1]:,} columns"
14    )
15
16    st.markdown("#### Column names and data types")
17    dtype_table = pd.DataFrame({
18        "Column": series.columns,
19        "Data Type": [str(dtype) for dtype in series.dtypes],
20        "Non-null Values": series.notna().sum().values,
21    })
22    st.dataframe(dtype_table, use_container_width=True, hide_index=True)
23
24    first_rows, last_rows = st.columns(2)
25    with first_rows:
26        st.markdown("#### First 5 observations")
27        st.dataframe(display_series.head(5), use_container_width=True, hide_index=True)
28    with last_rows:
29        st.markdown("#### Last 5 observations")
30        st.dataframe(display_series.tail(5), use_container_width=True, hide_index=True)
31
32    st.markdown("### 2. Missing Values and Duplicates")
33    missing_counts = series.isna().sum()
34    missing_table = pd.DataFrame({
35        "Column": series.columns,
36        "Missing Values": missing_counts.values,
37        "Missing (%)": (missing_counts / max(len(series), 1) * 100).round(2).values,
38    })
39    quality_columns = st.columns(3)
40    quality_columns[0].metric("Missing source target cells", f"{source_target_missing:,}")
41    quality_columns[1].metric("Duplicate source rows", f"{int(raw.duplicated().sum()):,}")
42    quality_columns[2].metric("Duplicate prepared dates", f"{int(series['Date'].duplicated().sum()):,}")
43    st.dataframe(missing_table, use_container_width=True, hide_index=True)
44    st.caption(
45        f"Missing target cells in the source row: {source_target_missing:,} of "
46        f"{len(source_target_numeric):,} ({source_target_missing_pct:.2f}%). "
47        "The preparation step excludes invalid dates and non-numeric/missing target values; it does not impute them."
48    )
49    if quality["raw_missing_cells"]:
50        st.caption(
51            f"The full wide-format source file also contains {quality['raw_missing_cells']:,} blank cells "
52            "across its other series/columns; the target-row check above isolates this analysis target."
53        )
54
55    st.markdown("### 3. Descriptive Statistics")
56    descriptive_stats = series[target_name].describe().to_frame("Tourist Arrivals")
57    descriptive_stats.index = [
58        "Count", "Mean", "Standard deviation", "Minimum", "25th percentile",
59        "Median", "75th percentile", "Maximum"
60    ]
61    st.dataframe(descriptive_stats, use_container_width=True)
62    st.caption(
63        "The mean is sensitive to unusually low or high months; compare it with the median and spread "
64        "when describing typical demand."
65    )
66
67    st.markdown("### 4. Distribution and Outlier Analysis")
68    target_values = series[target_name]
69    first_quartile = target_values.quantile(0.25)
70    third_quartile = target_values.quantile(0.75)
71    interquartile_range = third_quartile - first_quartile
72    outlier_lower = first_quartile - 1.5 * interquartile_range
73    outlier_upper = third_quartile + 1.5 * interquartile_range
74    outlier_mask = (target_values < outlier_lower) | (target_values > outlier_upper)
75    outlier_rows = series.loc[outlier_mask, ["Date", target_name]].copy()
76    outlier_rows["Date"] = outlier_rows["Date"].dt.strftime("%Y-%m")
77    distribution_columns = st.columns(2)

```

```

78     with distribution_columns[0]:
79         histogram = px.histogram(
80             series, x=target_name, nbins=24, title="Monthly arrivals distribution",
81             labels={target_name: "Tourist Arrivals", "count": "Months"}
82         )
83         st.plotly_chart(histogram, use_container_width=True)
84     with distribution_columns[1]:
85         distribution_box = px.box(
86             series, y=target_name, points="outliers", title="Overall arrivals box plot",
87             labels={target_name: "Tourist Arrivals"}
88         )
89         st.plotly_chart(distribution_box, use_container_width=True)
90     st.write(
91         f"The 1.5×IQR rule flags **{len(outlier_rows)} candidate outlier months** "
92         f"(below {outlier_lower:.0f} or above {outlier_upper:.0f}). These are candidates for investigation, "
93         f"not automatic errors or observations to remove."
94     )
95     if not outlier_rows.empty:
96         st.dataframe(outlier_rows, use_container_width=True, hide_index=True)
97
98     st.markdown("### 5. Time-Series Trend and Historical Demand")
99     trend_index = np.arange(len(series))
100    trend_slope, trend_intercept = np.polyfit(trend_index, target_values.to_numpy(), 1)
101    trend_series = series[["Date", target_name]].copy()
102    trend_series["Linear Trend"] = trend_intercept + trend_slope * trend_index
103    trend_chart = px.line(
104        trend_series, x="Date", y=[target_name, "Linear Trend"],
105        title="Monthly arrivals and fitted linear trend"
106    )
107    covid_start = pd.Timestamp("2020-02-01")
108    covid_end = pd.Timestamp("2022-03-31")
109    if series["Date"].min() <= covid_end and series["Date"].max() >= covid_start:
110        trend_chart.add_vrect(
111            x0=covid_start, x1=covid_end, fillcolor="#EF4444", opacity=0.12,
112            line_width=0, annotation_text="COVID-19 disruption",
113            annotation_position="top left", annotation_font=dict(color="#F87171", size=11)
114        )
115    trend_chart.update_layout(xaxis_title="Month", yaxis_title="Tourist Arrivals")
116    st.plotly_chart(trend_chart, use_container_width=True)
117    st.caption(
118        f"The fitted linear slope is {trend_slope:.0f} arrivals per observed month. "
119        f"It summarizes the full period and should be interpreted alongside the COVID-era shock and recovery."
120    )

```

Exact supplied app.py — source lines 481–600

```

1
2     st.markdown("### 6. Year-over-Year Analysis")
3     yearly = series_for_eda.groupby("Year", as_index=False).agg(
4         Annual_Total=(target_name, "sum"),
5         Monthly_Average=(target_name, "mean"),
6         Months_Observed=(target_name, "count"),
7     )
8     yearly["Year-over-Year Change (%)"] = yearly["Annual_Total"].pct_change().mul(100).round(2)
9     yearly_chart = px.bar(
10         yearly, x="Year", y="Annual_Total", title="Annual visitor arrivals",
11         labels={"Annual_Total": "Total Arrivals", "Year": "Year"},
12         hover_data=["Monthly_Average", "Months_Observed"]
13     )
14     st.plotly_chart(yearly_chart, use_container_width=True)
15     latest_year = int(yearly["Year"].iloc[-1])
16     latest_year_months = int(yearly["Months_Observed"].iloc[-1])
17     st.caption(
18         f"Annual totals are calculated from observed months. {latest_year} currently contains "
19         f"{latest_year_months} month(s), so it is a partial-year total and is not directly comparable with full years."
20     )
21     st.dataframe(yearly.round(2), use_container_width=True, hide_index=True)
22
23     st.markdown("### 7. Monthly Seasonality")
24     monthly_summary = series_for_eda.groupby("Month_Name", observed=False)[target_name].agg(
25         Mean="mean", Median="median", Standard_Deviation="std", Observations="count"
26     ).reset_index()
27     monthly_summary["Month_Name"] = monthly_summary["Month_Name"].astype(str)
28     monthly_chart = px.bar(
29         monthly_summary, x="Month_Name", y="Mean", title="Average arrivals by calendar month",
30         labels={"Month_Name": "Month", "Mean": "Average Arrivals"},
31         category_orders={"Month_Name": month_names}
32     )
33     st.plotly_chart(monthly_chart, use_container_width=True)
34     monthly_box = px.box(
35         series_for_eda, x="Month_Name", y=target_name, points="outliers",
36         title="Arrivals distribution by calendar month",
37         labels={"Month_Name": "Month", target_name: "Tourist Arrivals"},
38         category_orders={"Month_Name": month_names}
39     )
40     st.plotly_chart(monthly_box, use_container_width=True)
41     st.dataframe(monthly_summary.round(2), use_container_width=True, hide_index=True)
42
43     st.markdown("### 8. Year × Month Seasonality Heatmap")
44     heatmap_data = series_for_eda.pivot_table(
45         index="Year", columns="Month", values=target_name, aggfunc="mean"
46     ).reindex(columns=range(1, 13))

```

```

47 heatmap_data.columns = month_names
48 heatmap = px.imshow(
49     heatmap_data, aspect="auto", color_continuous_scale="YlOrRd",
50     labels={"x": "Month", "y": "Year", "color": "Arrivals"},
51     title="Monthly visitor arrivals by year"
52 )
53 st.plotly_chart(heatmap, use_container_width=True)
54 st.caption("Blank cells indicate months not present in the source data; color encodes arrivals.")
55
56 st.markdown("### 9. Rolling Averages")
57 rolling_series = series[["Date", target_name]].copy()
58 rolling_series["3-month rolling mean"] = target_values.rolling(3).mean()
59 rolling_series["12-month rolling mean"] = target_values.rolling(12).mean()
60 rolling_chart = px.line(
61     rolling_series, x="Date", y=[target_name, "3-month rolling mean", "12-month rolling mean"],
62     title="Observed arrivals with 3- and 12-month rolling means"
63 )
64 st.plotly_chart(rolling_chart, use_container_width=True)
65 st.caption("Rolling means smooth short-term variation; early values are blank until each window is available.")
66
67 st.markdown("### 10. Lag Relationships and Autocorrelation")
68 lag_columns = st.columns(2)
69 with lag_columns[0]:
70     lag_one_chart = px.scatter(
71         featured, x="Lag_1", y=target_name, title="Current arrivals vs previous month",
72         labels={"Lag_1": "Previous month's arrivals", target_name: "Current arrivals"},
73         trendline=None
74     )
75 st.plotly_chart(lag_one_chart, use_container_width=True)
76 with lag_columns[1]:
77     lag_twelve_chart = px.scatter(
78         featured, x="Lag_12", y=target_name, title="Current arrivals vs same month last year",
79         labels={"Lag_12": "Arrivals 12 months earlier", target_name: "Current arrivals"},
80         trendline=None
81     )
82 st.plotly_chart(lag_twelve_chart, use_container_width=True)
83
84 lag_correlations = {
85     lag: float(target_values.autocorr(lag=lag))
86     for lag in range(0, min(25, len(target_values)))
87 }
88 acf_data = pd.DataFrame({"Lag (months)": list(lag_correlations), "Correlation": list(lag_correlations.values())})
89 acf_chart = px.bar(
90     acf_data, x="Lag (months)", y="Correlation", title="Autocorrelation by monthly lag",
91     labels={"Correlation": "Autocorrelation"}
92 )
93 confidence_bound = 1.96 / np.sqrt(len(target_values))
94 acf_chart.add_hline(y=confidence_bound, line_dash="dash", line_color="#F87171")
95 acf_chart.add_hline(y=-confidence_bound, line_dash="dash", line_color="#F87171")
96 st.plotly_chart(acf_chart, use_container_width=True)
97 lag_one_correlation = lag_correlations.get(1, float("nan"))
98 lag_twelve_correlation = lag_correlations.get(12, float("nan"))
99 st.caption(
100     f"Descriptive autocorrelation: lag 1 = {lag_one_correlation:.3f}; "
101     f"lag 12 = {lag_twelve_correlation:.3f}. Dashed lines show the approximate ±1.96/√n reference; "
102     "these are exploratory guides, not a formal model significance test."
103 )
104
105 st.markdown("### 11. Feature Relationships")
106 correlation_columns = [
107     target_name, "Lag_1", "Lag_12", "Rolling_Mean_3", "Month_Sin", "Month_Cos"
108 ]
109 feature_correlations = featured[correlation_columns].corr()
110 correlation_chart = px.imshow(
111     feature_correlations, text_auto=".2f", zmin=-1, zmax=1, aspect="auto",
112     color_continuous_scale="RdBu_r", title="Target and selected feature correlations"
113 )
114 st.plotly_chart(correlation_chart, use_container_width=True)
115 st.caption("Correlations describe pairwise linear association and do not establish causation.")
116
117 st.markdown("### 12. EDA Findings and Feature-Engineering Rationale")
118 first_year_average = target_values.head(12).mean()
119 latest_year_average = target_values.tail(12).mean()
120 period_change = (latest_year_average / first_year_average - 1) * 100 if first_year_average else 0.0

```

Exact supplied app.py — source lines 601–720

```

1 highest_month = monthly_summary.loc[monthly_summary["Mean"].idxmax()]
2 lowest_month = monthly_summary.loc[monthly_summary["Mean"].idxmin()]
3 pre_covid = series.loc[series["Date"].between("2017-01-01", "2019-12-31"), target_name]
4 covid_period = series.loc[series["Date"].between(covid_start, covid_end), target_name]
5 if len(pre_covid) and len(covid_period) and pre_covid.mean():
6     covid_change = (covid_period.mean() / pre_covid.mean() - 1) * 100
7     covid_finding = (
8         f"Average arrivals during Feb 2020-Mar 2022 were {abs(covid_change):.1f}% "
9         f"{'lower' if covid_change < 0 else 'higher'} than the 2017-2019 average. "
10         "This real-world disruption is retained, not treated as an automatic data error."
11     )
12 else:
13     covid_finding = "The selected dataset does not contain both comparison periods needed to quantify the COVID-era change."
14
15 st.markdown(

```

```

16         f"- The mean of the latest 12 observations is **{period_change:+.1f}%** relative to the first 12, "
17         "summarizing long-term change while averaging over individual months.\n"
18         f"- The highest average calendar month is **{highest_month['Month_Name']}** "
19         f"({highest_month['Mean']:, .0f}); the lowest is **{lowest_month['Month_Name']}** "
20         f"({lowest_month['Mean']:, .0f}).\n"
21         f"- {covid_finding}\n"
22         f"- Lag correlations are **{lag_one_correlation:.3f}%** at one month and "
23         f"**{lag_twelve_correlation:.3f}%** at 12 months; these help assess whether recent and annual history "
24         "carry useful descriptive signal.\n"
25         f"- The IQR rule flags **{len(outlier_rows)}** candidate months. Investigate context before deciding "
26         "whether any value is erroneous; the EDA does not remove them."
27     )
28
29     feature_rationale = pd.DataFrame({
30         "EDA evidence": [
31             f"Month averages range from {lowest_month['Mean']:, .0f} to {highest_month['Mean']:, .0f} arrivals.",
32             f"One-month autocorrelation is {lag_one_correlation:.3f}.",
33             f"Twelve-month autocorrelation is {lag_twelve_correlation:.3f}.",
34             "Rolling means provide a smoothed summary of recent levels.",
35         ],
36         "Related model features": [
37             "Month_Sin, Month_Cos, Month, Quarter",
38             "Lag_1, Lag_2, Lag_3",
39             "Lag_12",
40             "Rolling_Mean_3, Rolling_Mean_6, Rolling_Mean_12",
41         ],
42     })
43     st.dataframe(feature_rationale, use_container_width=True, hide_index=True)
44     st.info(
45         "The baseline, polynomial trend experiments, and Random Forest are compared because the series "
46         "contains changing trend, monthly patterns, lag structure, and a major external shock. "
47         "Chronological test results and TimeSeriesSplit-not EDA alone-determine comparative model performance."
48     )
49
50     elif page == "Model Comparison":
51         st.title("Model Development & Evaluation")
52
53         st.markdown("### Test-set model comparison")
54         display_results = results.copy()
55         for c in ["MAE", "MSE", "RMSE"]:
56             display_results[c] = display_results[c].round(2)
57         display_results["R2"] = display_results["R2"].round(4)
58         st.dataframe(display_results, use_container_width=True, hide_index=True)
59
60         st.markdown("### Actual vs Predicted")
61         model_choice = st.selectbox(
62             "Choose a model",
63             ["Linear Regression", "Random Forest",
64              "Polynomial Regression (Degree 2)", "Polynomial Regression (Degree 4)"]
65         )
66         plot_df = actual_vs_pred[["Date", "Tourist_Arrivals", model_choice]].copy()
67         plot_df = plot_df.rename(columns={model_choice: "Predicted"})
68         fig = px.line(plot_df, x="Date", y=["Tourist_Arrivals", "Predicted"])
69         st.plotly_chart(fig, use_container_width=True)
70
71         st.markdown("### Random Forest Feature Importance")
72         rf_imp = random_forest_feature_importance(final_models["Random Forest"]).head(10)
73         fig_imp = px.bar(rf_imp.sort_values("Importance"), x="Importance", y="Feature", orientation="h")
74         st.plotly_chart(fig_imp, use_container_width=True)
75
76         st.markdown("### Time-Series Cross-Validation")
77         cv_show = cv.copy()
78         for c in ["MAE", "MSE", "RMSE"]:
79             cv_show[c] = cv_show[c].round(2)
80         cv_show["R2"] = cv_show["R2"].round(4)
81         st.dataframe(cv_show, use_container_width=True, hide_index=True)
82
83         summary = cv.groupby("Model")[["MAE", "RMSE", "R2"]].agg(["mean", "std"]).round(2)
84         st.markdown("### Cross-validation summary")
85         st.dataframe(summary, use_container_width=True)
86
87     elif page == "Forecast":
88         st.title("Future Tourist Demand Forecast")
89
90         c1, c2 = st.columns([1, 2])
91         model_name = c1.selectbox("Forecast Model", ["Linear Regression", "Random Forest"])
92         forecast_mode = c2.radio(
93             "Forecast selection",
94             ["Next N months", "Choose specific month(s)"],
95             index=1,
96             key="forecast_selection_mode",
97             horizontal=True,
98         )
99         last_observed_date = series["Date"].max()
100         available_forecast_dates = pd.date_range(
101             last_observed_date + pd.DateOffset(months=1), periods=12, freq="MS"
102         )
103         month_labels = {
104             date.strftime("%Y-%m"): f"(date.strftime('%B')) (Month {date.month:02d}, {date.year})"
105             for date in available_forecast_dates
106         }
107         date_by_month_label = {
108             label: pd.Timestamp(month_key + "-01")
109             for month_key, label in month_labels.items()
110         }

```

```

110     }
111     selected_month_labels = []
112     if forecast_mode == "Next N months":
113         horizon = c2.slider("Forecast Horizon (months)", 1, 12, 6)
114     else:
115         selected_month_labels = c2.multiselect(
116             "Select target month(s)", list(month_labels.values())
117         )
118     c2.caption(
119         "Forecasts are generated sequentially; unselected months are modeled internally "
120         "when needed to reach a later selected month."

```

Exact supplied app.py — source lines 721–819

```

1         )
2
3     st.caption(
4         "Forecasting is recursive: after the first predicted month, the prediction is used "
5         "as historical input for later months."
6     )
7
8     if st.button("Generate Forecast", type="primary"):
9         if forecast_mode == "Choose specific month(s)" and not selected_month_labels:
10             st.warning("Choose at least one target month before generating a forecast.")
11         else:
12             selected_dates = [date_by_month_label[label] for label in selected_month_labels]
13             forecast_horizon = horizon if forecast_mode == "Next N months" else max(
14                 (date.year - last_observed_date.year) * 12 + date.month - last_observed_date.month
15                 for date in selected_dates
16             )
17             forecast_path = recursive_forecast(
18                 final_models[model_name], series, months=forecast_horizon
19             )
20             forecast = forecast_path
21             if forecast_mode == "Choose specific month(s)":
22                 forecast = forecast[forecast["Date"].isin(selected_dates)].reset_index(drop=True)
23             st.session_state["forecast"] = forecast
24             st.session_state["forecast_path"] = forecast_path
25             st.session_state["forecast_selected_dates"] = selected_dates
26             st.session_state["forecast_model"] = model_name
27             st.session_state["forecast_mode"] = forecast_mode
28
29     if "forecast" in st.session_state:
30         forecast = st.session_state["forecast"]
31         st.success(f"Forecast generated using {st.session_state['forecast_model']}.")
32         forecast_table = pd.DataFrame({
33             "Forecast Month": forecast["Date"].dt.strftime("%B (Month %m), %Y"),
34             "Predicted Arrivals": forecast["Predicted_Tourist_Arrivals"].map(format_indian_number),
35             "Arrivals in Words": forecast["Predicted_Tourist_Arrivals"].map(number_to_indian_words),
36         })
37         st.dataframe(forecast_table, use_container_width=True, hide_index=True)
38
39         history_plot = series.tail(36)[["Date", "Tourist_Arrivals"]].copy()
40         history_plot["Type"] = "Historical"
41         forecast_path = st.session_state.get("forecast_path", forecast)
42         forecast_path_plot = forecast_path.rename(
43             columns={"Predicted_Tourist_Arrivals": "Tourist_Arrivals"}
44         )[["Date", "Tourist_Arrivals"]].copy()
45         forecast_path_plot["Type"] = "Forecast path"
46         forecast_anchor = history_plot.tail(1).copy()
47         forecast_anchor["Type"] = "Forecast path"
48         forecast_path_plot = pd.concat(
49             [forecast_anchor, forecast_path_plot], ignore_index=True
50         )
51         combined = pd.concat([history_plot, forecast_path_plot], ignore_index=True)
52
53         fig = px.line(
54             combined, x="Date", y="Tourist_Arrivals", color="Type",
55             title="Recent History + Forecast Path", markers=True,
56             color_discrete_map={
57                 "Historical": "#82C8FF",
58                 "Forecast path": "#0B84F3",
59             }
60         )
61         selected_forecast_dates = st.session_state.get("forecast_selected_dates", [])
62         selected_forecast_points = forecast_path[
63             forecast_path["Date"].isin(selected_forecast_dates)
64         ]
65         if not selected_forecast_points.empty:
66             fig.add_scatter(
67                 x=selected_forecast_points["Date"],
68                 y=selected_forecast_points["Predicted_Tourist_Arrivals"],
69                 mode="markers",
70                 name="Selected target month(s)",
71                 marker=dict(color="#FF6B6B", size=11, symbol="diamond"),
72                 hovertemplate="%{x|B %Y}<br>Selected forecast: %{y:,.0f}<extra></extra>",
73             )
74         fig.update_layout(xaxis_title="Date", yaxis_title="Tourist Arrivals")
75         st.plotly_chart(fig, use_container_width=True)
76         if selected_forecast_dates:
77             st.caption(
78                 "The blue line shows every recursively forecast month; red diamonds mark your selected target month(s)."

```

```

79         )
80
81     total_forecast = forecast["Predicted_Tourist_Arrivals"].sum()
82     avg_forecast = forecast["Predicted_Tourist_Arrivals"].mean()
83     a, b = st.columns(2)
84     if len(forecast) == 1:
85         selected_month = forecast["Date"].iloc[0]
86         a.metric(
87             "Selected Month Forecast",
88             format_indian_number(forecast["Predicted_Tourist_Arrivals"].iloc[0])
89         )
90         a.caption(
91             f"In words: {number_to_indian_words(forecast['Predicted_Tourist_Arrivals'].iloc[0])} visitors"
92         )
93         b.metric("Forecast Month", selected_month.strftime("%B (Month %m), %Y"))
94     else:
95         a.metric("Forecast Total", format_indian_number(total_forecast))
96         a.caption(f"In words: {number_to_indian_words(total_forecast)} visitors")
97         b.metric("Average Monthly Forecast", format_indian_number(avg_forecast))
98         b.caption(f"In words: {number_to_indian_words(avg_forecast)} visitors")

```

17. Limitations, Risks and Future Improvements

- Only historical visitor-arrival information is used directly; external drivers such as airline capacity, GDP, exchange rates, hotel occupancy, holidays/events, weather and policy changes are not explicit model inputs.
- The COVID-era structural break is large. A single global regression relationship may not represent every regime equally well.
- Recursive forecasting can accumulate errors because later predictions use earlier predictions.
- The current forecast output is a point estimate without prediction intervals.
- The dataset has 217 monthly observations in the supplied snapshot, which is modest for deep-learning models.
- Random Forest may be less effective for extrapolating long-term trend outside its training range.
- 2026 is partial in the local dataset snapshot, so year totals for 2026 are not directly comparable with full years.

17.1 Useful alternative models

Alternative	Core idea	Where it could help
Seasonal Naive	Forecast $y(t) = y(t-12)$	Very strong baseline for monthly seasonality.
Moving Average	Recent mean	Simple smoothing baseline.
Holt / Holt-Winters	Trend + seasonality	Classical forecasting.
ARIMA / SARIMA	Autoregressive + differencing + seasonality	Classical time-series modeling.
SARIMAX	SARIMA + external variables	Useful when economic/flight/holiday drivers are added.
Ridge / Lasso / ElasticNet	Regularized linear models	Can reduce coefficient instability and feature redundancy.
Gradient Boosting / XGBoost / LightGBM	Boosted trees	Strong nonlinear tabular alternatives.
SVR	Margin-based nonlinear regression	Can work on smaller datasets after scaling.
LSTM / GRU / Transformers	Deep sequence models	Possible for larger datasets; likely overkill for this 217-point series.

Appendix A. Formula Sheet

Concept	Formula / definition
Mean	$\bar{x} = (1/n) \sum x_i$
Median	Middle value after sorting.
Sample standard deviation	$s = \sqrt{[\sum (x_i - \bar{x})^2 / (n-1)]}$
Variance	s^2
Rolling mean	$RM_{\square}(t) = (1/k) \sum_{i=t-k}^t y_{\square-i}$
Lag	$Lag_{\square}(t) = y_{\square-t}$
Cyclical month sine	$\sin(2\pi \cdot \text{Month}/12)$
Cyclical month cosine	$\cos(2\pi \cdot \text{Month}/12)$
Linear Regression	$\hat{y} = \beta_0 + \sum \beta_{\square} x_{\square}$
MAE	$(1/n) \sum y - \hat{y} $
MSE	$(1/n) \sum (y - \hat{y})^2$
RMSE	$\sqrt{\text{MSE}}$
R ²	$1 - \sum (y - \hat{y})^2 / \sum (y - \bar{y})^2$
IQR	$Q3 - Q1$
IQR lower fence	$Q1 - 1.5 \times \text{IQR}$
IQR upper fence	$Q3 + 1.5 \times \text{IQR}$
Approx. ACF reference	$\pm 1.96/\sqrt{n}$

Units

- MAE: visitors.
- MSE: visitor².
- RMSE: visitors.
- R²: unitless.
- Feature importance: normalized model-specific importance values, unitless.

Appendix B. Source-Code Appendix and Run Instructions

B.1 Environment setup

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install -r requirements.txt
4 python3 analysis.py
5 streamlit run app.py
```

The project README specifies these commands. The supplied VS Code screenshot shows a .venv environment and requirements.txt. The exact requirements file contents were not available in the materialized project snapshot.

B.2 Key ML code excerpts

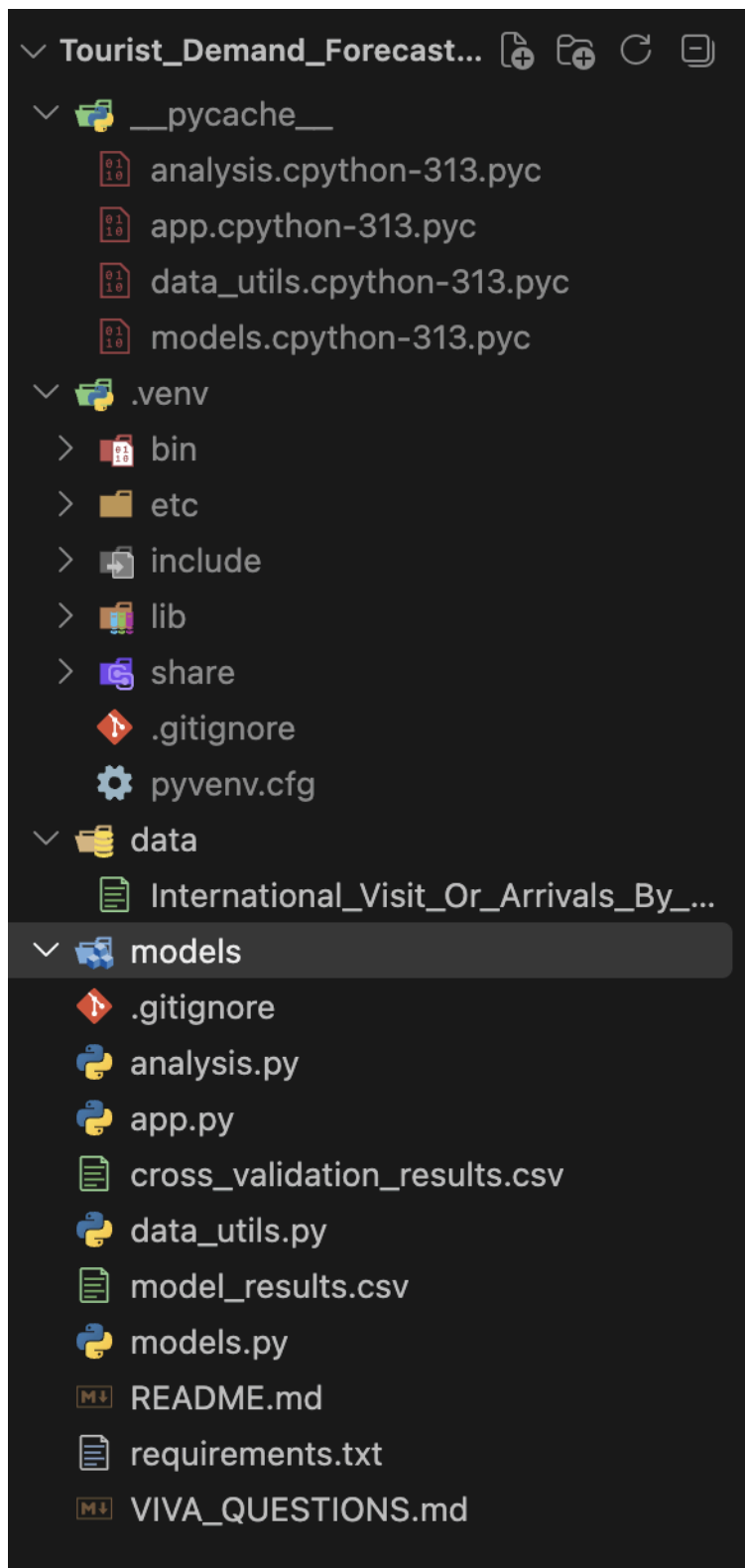
```
1 # Chronological split
2 split = int(len(featured_df) * (1 - test_size))
3 train_df = featured_df.iloc[:split].copy()
4 test_df = featured_df.iloc[split:].copy()
5
6 # Metrics
7 mse = mean_squared_error(y_true, y_pred)
8 mae = mean_absolute_error(y_true, y_pred)
9 rmse = np.sqrt(mse)
10 r2 = r2_score(y_true, y_pred)
11
12 # TimeSeriesSplit
13 tscv = TimeSeriesSplit(n_splits=5)
14 for train_idx, test_idx in tscv.split(X):
15     model.fit(X.iloc[train_idx], y.iloc[train_idx])
16     pred = model.predict(X.iloc[test_idx])
```

B.3 Why each code block exists

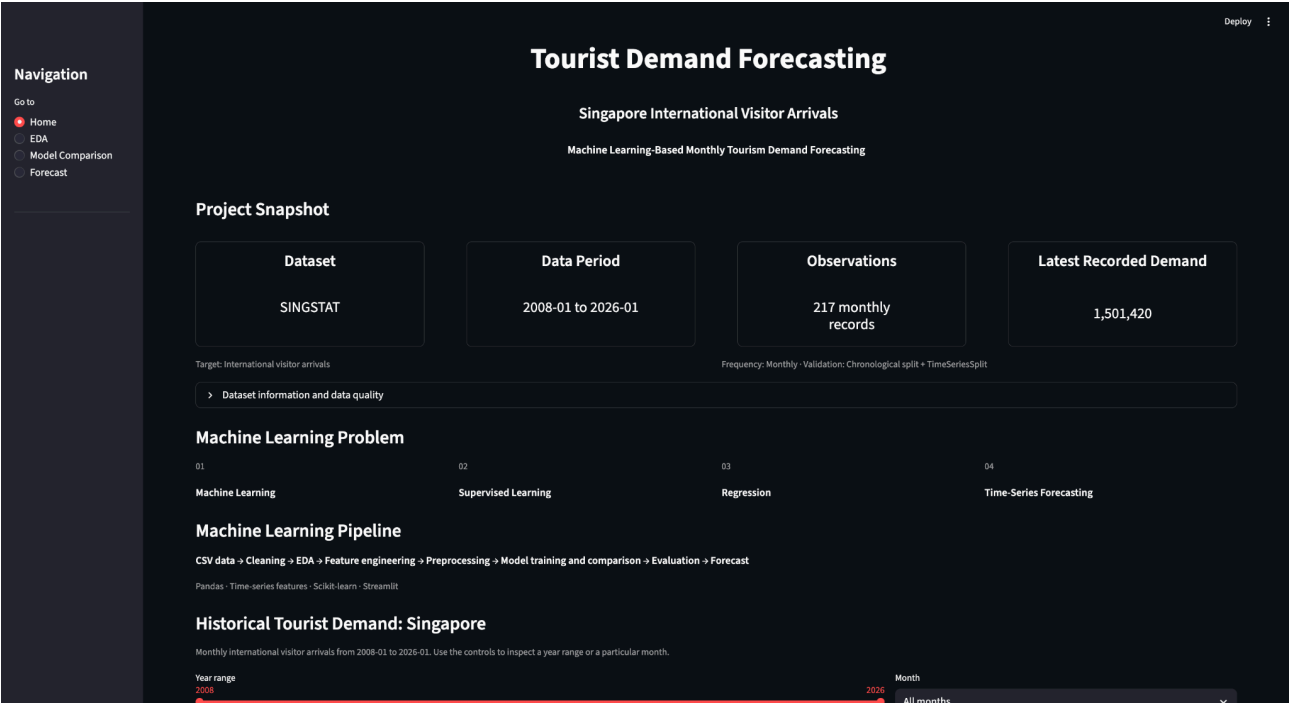
Code block	Purpose
Load	Bring the official CSV into pandas.
Prepare total series	Convert wide monthly columns into a chronological target series.
add features	Expose time, seasonality, lags and smoothed history.
chronological_split	Simulate realistic future prediction.
build_linear_model	Scale numeric features + one-hot calendar categories + linear model.
build_random_forest	Create nonlinear ensemble benchmark.
build_polynomial_model	Create degree-2 and degree-4 curve-fitting experiments.
evaluate	Make model results comparable.
time_series_cv	Check stability across multiple historical validation windows.
recursive_forecast	Generate multi-month future predictions.
Streamlit app	Make all analysis and forecasts interactive.

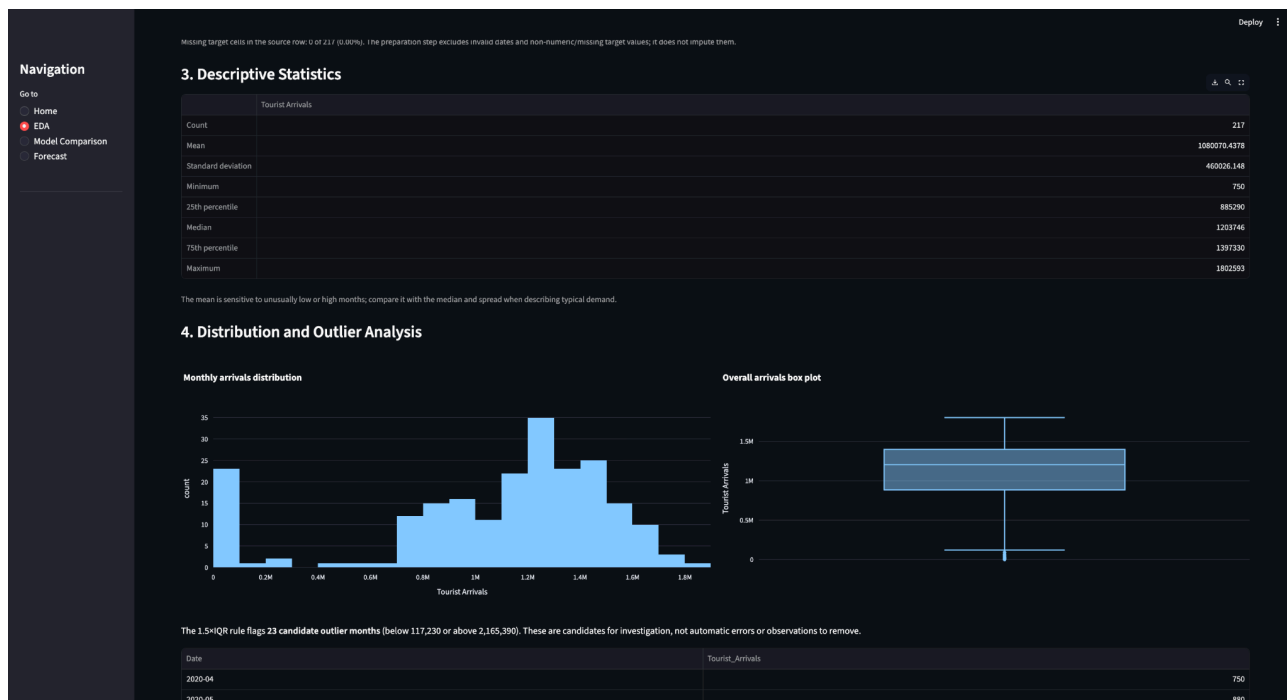
Appendix C. Project Screenshots

The following screenshots were supplied with the project and are included as evidence of the working Streamlit interface and VS Code project structure.

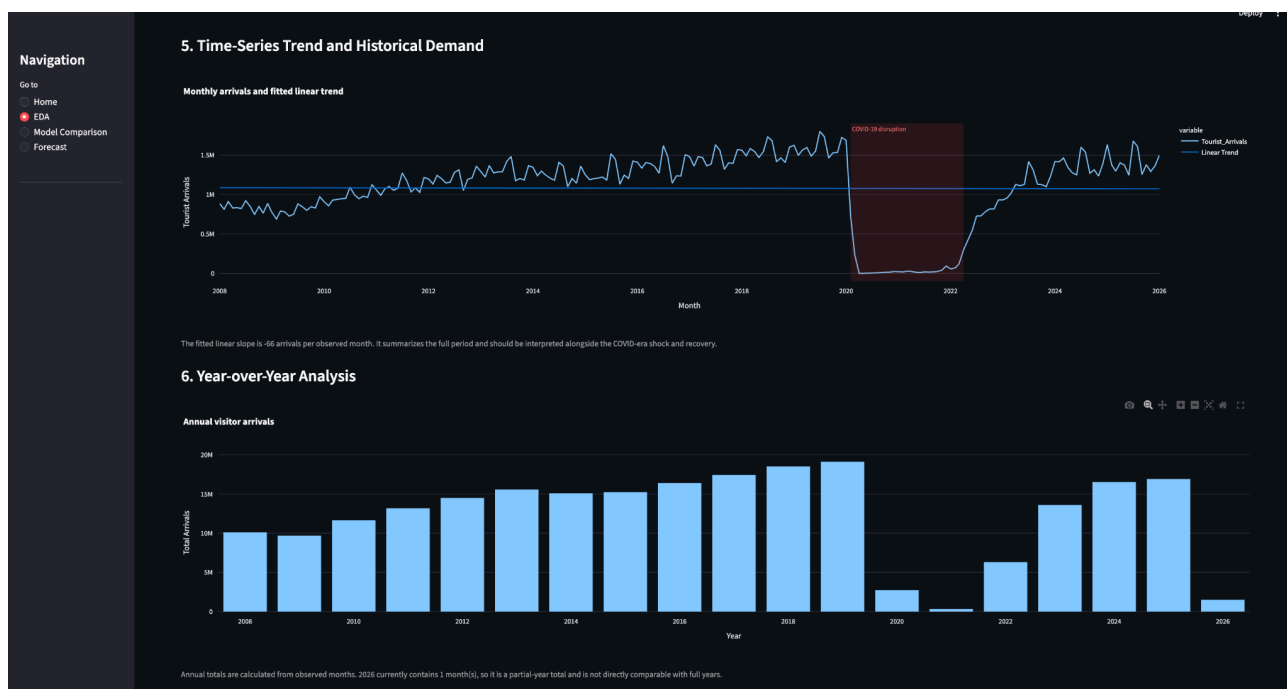


VS Code project structure

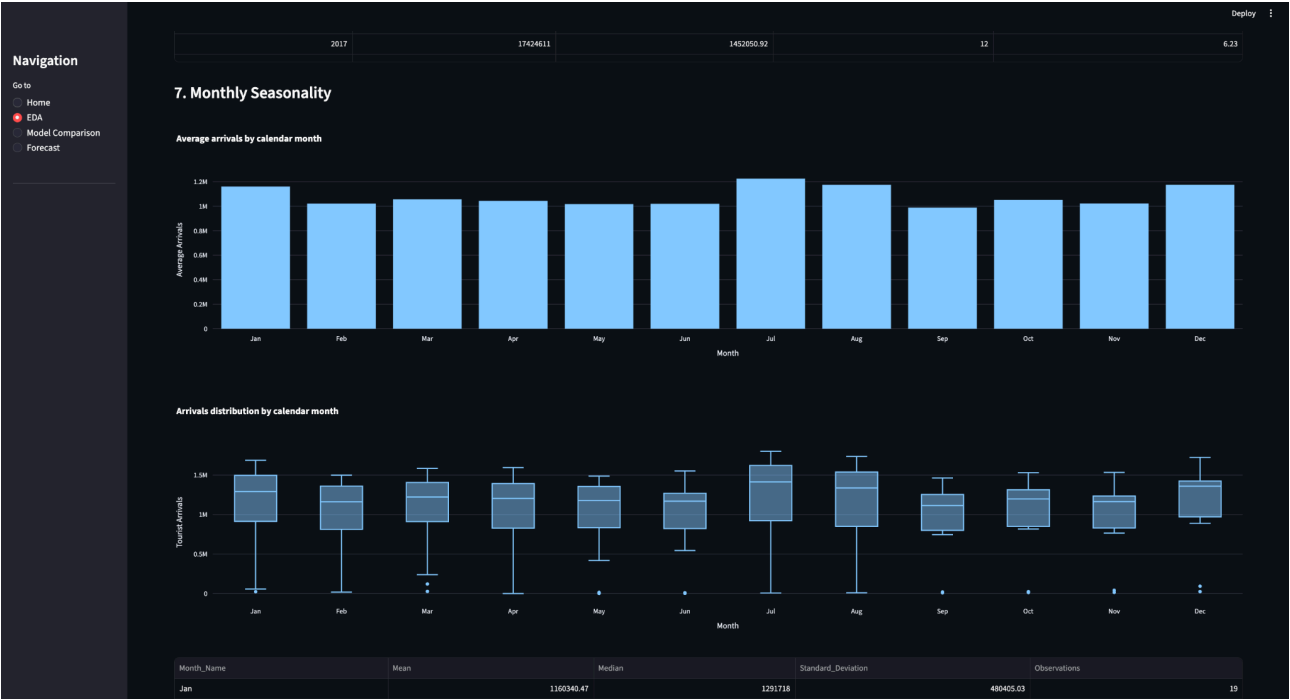




Distribution and outlier analysis



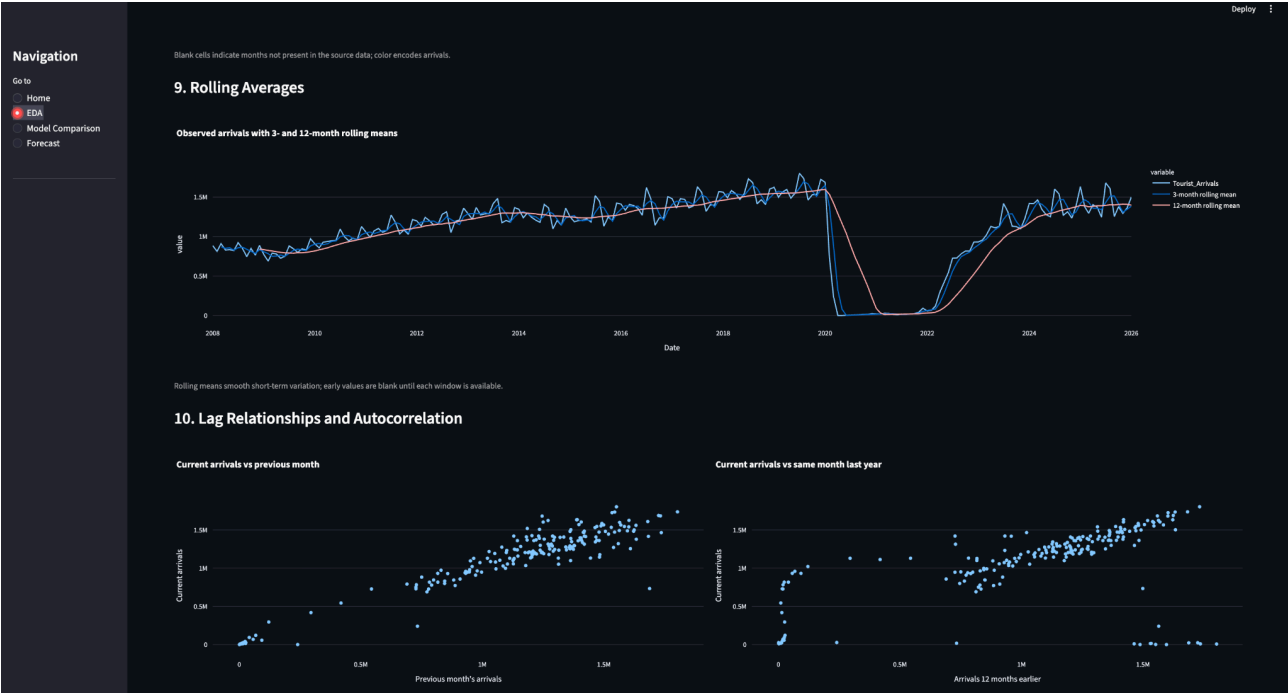
Trend and year-over-year analysis



Monthly seasonality



Year x Month heatmap



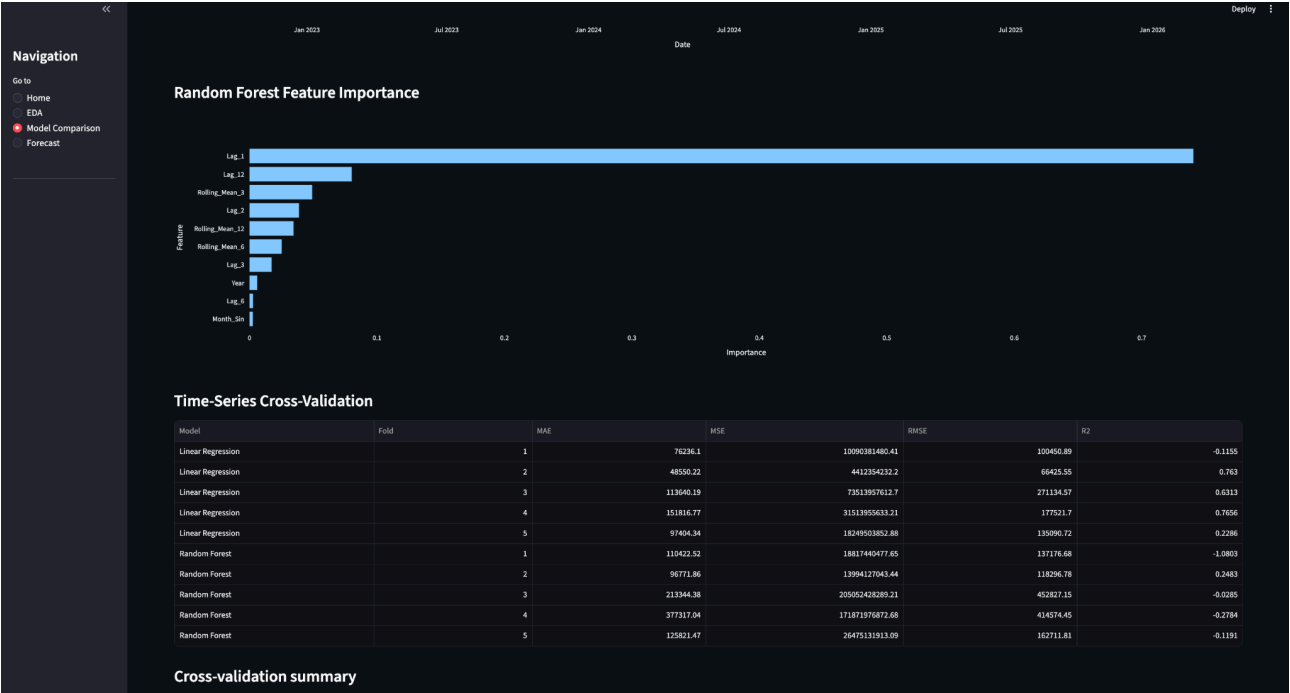
Rolling averages and lag relationships



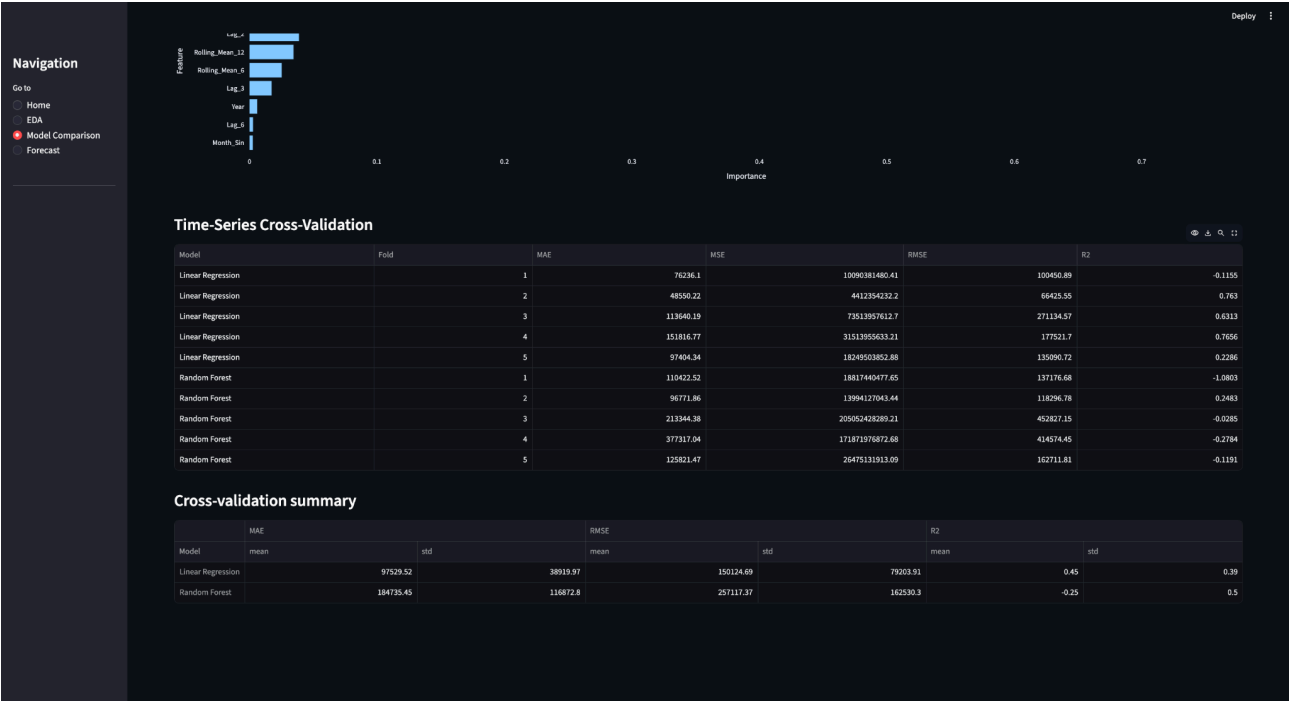
Autocorrelation / feature relationships



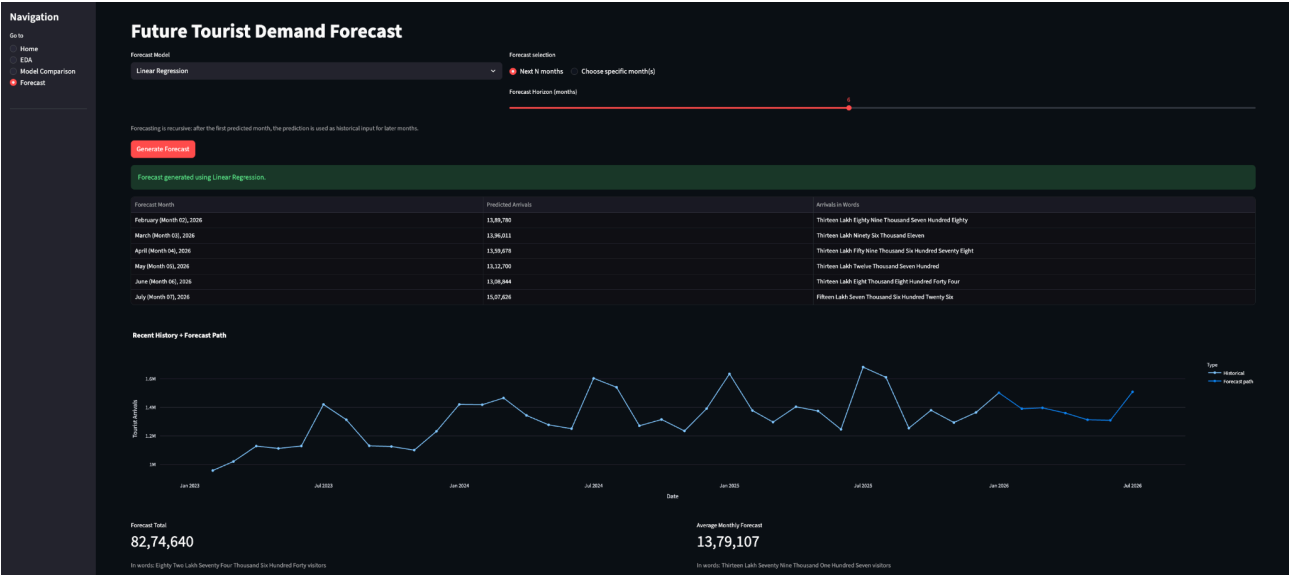
Model comparison



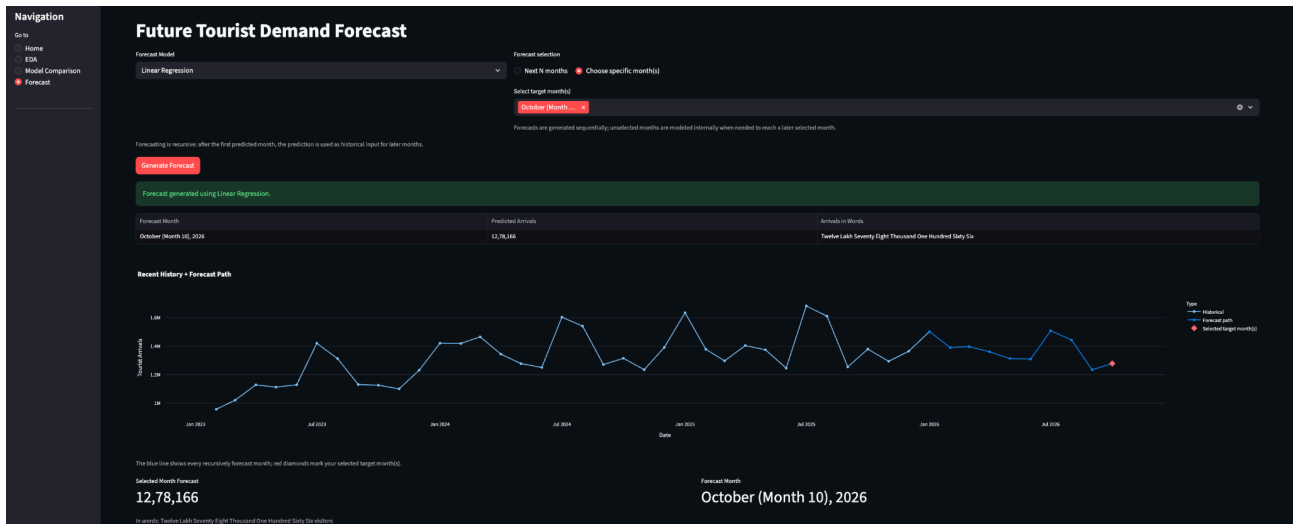
Random Forest feature importance



TimeSeriesSplit / CV summary



Six-month forecast



Specific-month forecast

References and Evidence

1. Singapore Department of Statistics (SINGSTAT). International Visitor Arrivals By Place Of Residence, Monthly. data.gov.sg. Official dataset page:
https://data.gov.sg/datasets/d_7e7b2ee60c6ffc962f80fef129cf306e/view
2. Supplied project source files: data_utils.py, models.py, analysis.py, app.py.
3. Supplied generated outputs: model_results.csv and cross_validation_results.csv.
4. Supplied project documents: README.md, FINAL_REPORT_DRAFT.md, PRESENTATION_OUTLINE.md and VIVA_QUESTIONS.md.
5. Scikit-learn concepts used by the implementation: LinearRegression, PolynomialFeatures, RandomForestRegressor, TimeSeriesSplit, StandardScaler, OneHotEncoder and regression metrics.